

# Balancing Data for Anomaly-based Intrusion Detection Systems (IDS)

**Muditha Cherangani**

**Supervisor :** Krzysztof Bartoszek

**Examiner :** Frank Miller

**External supervisor :** Arkadiusz Warzyński

## Upphovsrätt

Detta dokument hålls tillgängligt på Internet – eller dess framtida ersättare – under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <https://ep.liu.se/>.

## Copyright

The publishers will keep this document online on the Internet – or its possible replacement – for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: <https://ep.liu.se/>.

© Muditha Cherangani

## Abstract

Intrusion Detection Systems (IDS) are critical for cybersecurity, yet their performance is frequently compromised by class imbalance in datasets, where underrepresented attack classes lead to degraded detection rates. This study rigorously evaluates machine learning-based Network Intrusion Detection Systems (NIDS) using the CICIDS2017 dataset, addressing class imbalance through Synthetic Minority Oversampling Technique (SMOTE). This study systematically evaluates five supervised models Logistic Regression, k-Nearest Neighbors, Naive Bayes, Random Forest, and Deep Neural Networks for binary classification, along with two unsupervised approaches (K-Means and Isolation Forest), across seven randomly selected attack sample instances from the training set. Controlled imbalance experiments, where the attack sample size as a proportion of the normal data in the training set is progressively increased (1%, 5%, 10%, 30%, 60%, 80%, 100%) to analyze their impact on model performance.

This study analysis leverages balanced accuracy, precision, recall, F1-score, and Matthews Correlation Coefficient (MCC) to quantify performance. Results reveal that Random Forest dominates supervised methods, achieving near-flawless detection on balanced data. In stark contrast, unsupervised techniques exhibit severe limitations. K-Means and Isolation Forest fail to generalize beyond rare anomalies.

This study provides valuable insights into optimizing NIDS with machine learning, emphasizing the importance of data balancing and algorithm selection for robust cybersecurity solutions. The findings suggest that SMOTE-enhanced supervised models, particularly Random Forest, offer the most reliable intrusion detection, while unsupervised techniques require further refinement for practical deployment.

## Acknowledgements

I begin by expressing my deepest thanks to Linköping University for the invaluable opportunity to engage in this Master's program and thesis, an experience that has shaped me in countless ways.

I extend my sincerest thanks to my university supervisor, Professor Krzysztof Bartoszek, for his exceptional guidance and unwavering support throughout this thesis. His insightful discussions and fresh perspectives greatly influenced the ideas presented in this work. Each conversation with him reinvigorated my motivation, pushing me to explore new approaches that ultimately led to significant progress.

I am also profoundly grateful to my external supervisor, Professor Arkadiusz Warzyński from Wrocław University of Science and Technology, Poland for his constant encouragement and invaluable contributions to this thesis. His boundless enthusiasm and dedication were truly inspiring, fueling my passion and commitment to this work.

My heartfelt thanks go to Professor Frank Miller for his constructive feedback and for taking the time to review my thesis, helping me refine it further.

Above all, I owe immense gratitude to my family, Amma, Thaththa, and Sister, for their endless love and support, which words cannot adequately express. To all the wonderful friends I met here, the people I lived with, and everyone who made these years unforgettable a thousand thanks!

# Contents

## Table of Contents

|                                                            |      |
|------------------------------------------------------------|------|
| Abstract .....                                             | iii  |
| Acknowledgements.....                                      | iv   |
| Contents .....                                             | v    |
| Glossary / List of acronyms .....                          | vi   |
| List of Figures .....                                      | vii  |
| List of Table.....                                         | viii |
| 1. Introduction.....                                       | 1    |
| 1.1 Objective .....                                        | 3    |
| 1.2 Generative Artificial Intelligence.....                | 3    |
| 2. Related Work .....                                      | 4    |
| 3. Data.....                                               | 6    |
| 3.1 Network Traffic Types.....                             | 7    |
| 3.2 Explanation of Network Traffic Features .....          | 9    |
| 4. Theory .....                                            | 13   |
| 4.1 Supervised Learning .....                              | 13   |
| 4.2 Unsupervised Learning .....                            | 13   |
| 4.3 Sampling .....                                         | 14   |
| 4.4 Synthetic Minority Oversampling Technique (SMOTE)..... | 15   |
| 4.5 Deep Neural Network (DNN).....                         | 17   |
| 4.6 K-Nearest Neighbors (KNN) .....                        | 19   |
| 4.7 Naive Bayes .....                                      | 20   |
| 4.8 Random Forest .....                                    | 21   |
| 4.9 Logistic Regression.....                               | 21   |
| 4.10 K – Means.....                                        | 22   |
| 4.11 Isolation Forest.....                                 | 22   |
| 4.12 Evaluation Metrics .....                              | 23   |
| 5. Methodology .....                                       | 26   |
| 5.1 Data Preprocessing.....                                | 26   |
| 5.2 Feature Selection.....                                 | 27   |
| 5.3 Dataset Spitting.....                                  | 28   |
| 5.4 Training Dataset.....                                  | 28   |
| 5.5 Test Dataset.....                                      | 31   |

|                                                 |    |
|-------------------------------------------------|----|
| 5.6 Compare Performance Metrics of Models ..... | 32 |
| 6. Results.....                                 | 33 |
| 6.1 Before and after SMOTE apply .....          | 33 |
| 6.2 Supervised Learning Results.....            | 34 |
| 6.3 Unsupervised Learning Results .....         | 39 |
| 7. Discussion .....                             | 43 |
| 8. Conclusion .....                             | 48 |
| Ethical Considerations .....                    | 50 |
| Bibliography .....                              | 51 |

## **Glossary / List of acronyms**

IDS - Intrusion Detection Systems

NIDS - Network Intrusion Detection Systems

CICIDS - Canadian Institute for Cybersecurity Intrusion Detection Systems

SMOTE - Synthetic Minority Oversampling Technique

ML - Machine Learning

DNN – Deep Neural Network

KNN - K-Nearest Neighbors,

MCC - Matthews Correlation Coefficient

## List of Figures

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| 3.1 Part of Dataset.....                                                             | 12 |
| 4.1 The schematic of SMOTE algorithm.....                                            | 15 |
| 4.2 General Architecture for a Deep Neural Network with Two Hidden Layers.....       | 17 |
| 5.1 Selected Features.....                                                           | 27 |
| 5.2 Class Distribution with Percentage Level.....                                    | 28 |
| 5.3 Proposed Methodological Supervised Learning binary classification Framework..... | 32 |
| 6.1 Class Distribution Before and After SMOTE.....                                   | 33 |
| 6.2 DNN Model Performance.....                                                       | 34 |
| 6.3 KNN Model Performance.....                                                       | 35 |
| 6.4 Naive Bayes Model Performance.....                                               | 36 |
| 6.5 Random Forest Model Performance.....                                             | 37 |
| 6.6 Logistic Regression Model Performance.....                                       | 38 |
| 6.7 K-Means Benign Traffic Model Performance.....                                    | 39 |
| 6.8 K-Means Attack Traffic Model Performance.....                                    | 40 |
| 6.9 Isolation Forest Benign Traffic Model Performance.....                           | 41 |
| 6.10 Isolation Forest Attack Traffic Model Performance.....                          | 42 |

## List of Table

|                                                                                        |    |
|----------------------------------------------------------------------------------------|----|
| 1.1 Machine learning classification methods for anomaly-based intrusion detection..... | 1  |
| 2.1 Summary of Related Works with CICIDS2017 dataset.....                              | 5  |
| 3.1 Days with Traffic Type.....                                                        | 6  |
| 3.2 Traffic Types with Observations and Percentage.....                                | 7  |
| 4.1 Confusion Matrix.....                                                              | 23 |
| 5.1 Percentage Level.....                                                              | 28 |
| 5.2 DNN Hyperparameter.....                                                            | 30 |
| 5.3 KNN Hyperparameter.....                                                            | 30 |
| 5.4 Naive Bayes Hyperparameter.....                                                    | 30 |
| 5.5 Random Forest Hyperparameter.....                                                  | 30 |
| 5.6 Logistic Regression Hyperparameter.....                                            | 31 |
| 5.7 K-Means Hyperparameter.....                                                        | 31 |
| 5.8 Isolation Forest Hyperparameter.....                                               | 31 |

# 1. Introduction

An Intrusion Detection System (IDS) is a security mechanism designed to monitor network traffic or system activities for malicious actions or policy violations. Its primary purpose is to detect and alert administrators about potential security breaches, unauthorized access, or anomalous behavior that could indicate a cyber-attack. IDS plays a critical role in maintaining information systems' integrity, confidentiality, and availability [23]. Based on the detection mechanism, Intrusion Detection Systems are categorized into two main types: *Misuse-Based IDS* and *Anomaly-Based IDS*. Misuse-based IDS is limited to detecting known attacks using predefined signatures, while anomaly-based IDS can identify new or unusual behaviors, offering better protection against emerging threats [24]. At the core of any anomaly-based intrusion detection system lies its Machine Learning (ML) component, which enables the automated identification of malicious patterns in network traffic. The subsequent section details the implementation of key ML techniques within this study's IDS framework, focusing on their specific roles in detecting deviations from normal behavior [23].

| <b>Supervised Learning</b> | <b>Unsupervised Learning</b> | <b>Probabilistic Learning</b> | <b>Soft Computing</b> | <b>Combination Learners</b> |
|----------------------------|------------------------------|-------------------------------|-----------------------|-----------------------------|
| Parametric                 | Clustering                   | Naïve bayes                   | ANN based             | Ensemble based              |
| Non-parametric             | Association mining           | Bayesian net                  | Fuzzy logic           | Fusion based                |
|                            | Outlier mining               |                               |                       | Hybrid                      |

**Table 1.1:** Machine learning classification methods for anomaly-based intrusion detection.

Recent advances in intrusion detection have led to diverse machine learning classification methods for IDS design. As outlined in Table 1.1 [25], current taxonomies identify five principal ML techniques suitable for anomaly-based detection systems. These approaches fundamentally differ in their handling of training data and parameter optimization. Within IDS architectures, supervised learning methods operate on the critical assumption that the dataset contains clearly labeled instances, where each network flow or packet is classified as either an attack or not. These supervised techniques can be further categorized into parametric models and non-parametric models. Parametric models: characterized by a fixed set of parameters that remain constant regardless of dataset size. Examples: Logistic Regression, Naïve Bayes. Non-Parametric models: employ flexible parameters that scale with dataset complexity. Examples: k-Nearest Neighbors (KNN), Decision Trees, Support Vector Machines (SVM), Neural Networks [23].

Unsupervised learning offers critical advantages for IDS by functioning without labeled data. Clustering methods (e.g., K-Means) autonomously group similar traffic patterns, while outlier detection (e.g., Isolation Forest) identifies deviations, enabling discovery of both novel attack patterns and subtle anomalies that supervised methods might miss [23]. Probabilistic learning approaches provide a robust framework for intrusion detection by explicitly modeling the inherent randomness and uncertainty in network traffic patterns [26]. These methods dynamically update their probability estimates as new evidence emerges from training data, allowing them to adapt to evolving attack signatures (e.g., Naïve Bayes) [27]. The Deep Neural Network implementation specifically embodies soft computing principles through its probabilistic feature learning and adaptive decision boundaries [23]. Ensemble learning frameworks architecturally combine multiple base learners, creating a meta-classifier that capitalizes on model diversity to achieve more accurate and stable predictions than any individual component (e.g., Random Forest). This proves

particularly valuable in intrusion detection, where different algorithms may excel at identifying distinct attack characteristics [23].

IDS are broadly classified into three categories based on data collection methods: Network-based IDS (NIDS), Host-based IDS (HIDS), and Web-based IDS (WIDS) [23]. This study focuses on NIDS. NIDS monitors network traffic in real-time, analyzing packets across entire segments to detect threats like DDoS attacks and port scanning. It is typically deployed at strategic network points, such as gateways or between subnets, providing broad visibility but limited insight into encrypted traffic. HIDS operates at the host level, installed directly on endpoints like servers and workstations to monitor system logs, file integrity, and user activities, making it effective against insider threats and malware. WIDS specializes in protecting web applications by inspecting HTTP/HTTPS traffic for attacks such as SQL injection and cross-site scripting (XSS). While NIDS offers network-wide coverage, HIDS provides deep endpoint visibility, and WIDS secures application-layer vulnerabilities. Together, these systems form a layered defense strategy, with NIDS detecting external threats, HIDS identifying compromised hosts, and WIDS preventing web-specific exploits [4].

Effective anomaly detection models depend on quality training data, available either as public or private datasets [23]. Public datasets enable standardized comparison of detection methods, with the Canadian Institute for Cybersecurity IDS 2017 (CICIDS2017) [6] offering lab-generated but realistic attack simulations covering modern threats. This CICIDS2017 dataset is imbalanced, with a natural class imbalance between normal network traffic (Benign) and attack instances. Imbalanced datasets pose significant challenges for IDS classifiers, as the disproportionate ratio of normal to attack instances biases models toward majority class prediction [4]. Synthetic Minority Oversampling Technique (SMOTE) addresses this by synthetically generating minority class samples through k-nearest neighbor interpolation in feature space, a solution for imbalanced datasets [16]. This study evaluates model performance using key classification metrics, including balanced accuracy, precision, recall, F1-score, and Matthews Correlation Coefficient (MCC) [16]. This study addresses Binary Classification [4], which refers to a supervised learning task where instances are categorized into exactly one of two distinct classes or groups (e.g., Benign vs Attack).

## 1.1 Objective

This study investigates critical challenges in intrusion detection research, particularly the natural class imbalance between normal network traffic (Benign) and attack instances in the CICIDS2017 dataset, which realistically reflects infrastructure. The study uses binary classification (*Benign vs. Attack traffic*) to determine robustness under varying imbalance conditions. Supervised (Logistic Regression, KNN, Naive Bayes, Random Forest, Deep Neural Networks) and unsupervised (K-Means, Isolation Forest) learning approaches, assessing their effectiveness via balanced accuracy, precision, recall, F1-score, and MCC. Controlled imbalance experiments, where the attack sample size as a proportion of the normal data in the training set is progressively increased (1%, 5%, 10%, 30%, 60%, 80%, 100%), to analyze their impact on model performance. Supervised learning is ideal for intrusion detection when labeled data (normal vs. attack traffic) is available. It allows models to learn explicit patterns from historical data, making it effective for detecting known attack types. Unsupervised methods are valuable for detecting novel or zero-day attacks without labeled data. They identify anomalies by modeling normal behavior or clustering deviations.

### Research Questions

1. From the set of considered machine learning methods (Logistic Regression, K-Nearest Neighbors, Naive Bayes, Random Forest, Deep Neural Networks, K-Means, Isolation Forest), which generalizes the best in imbalanced intrusion detection environments?
2. How do different attack proportions (1%–100%) influence model performance metrics?
3. Which attack sample sizes (quantified as proportions of normal network traffic) are necessary such that supervised and unsupervised models perform well??

By addressing these questions, this study aims to provide actionable insights for developing more resilient intrusion detection systems under real-world data imbalance scenarios.

## 1.2 Generative Artificial Intelligence

*Grammarly*, an English-language writing assistant tool, was used in this study to review spelling, grammar, and tone.

## 2. Related Work

This paper's authors [2] address the challenges of improving the performance of an AdaBoost-based IDS using the CICIDS 2017 dataset. Previous research highlighted limitations such as imbalanced training data and suboptimal classification methods, leading to inferior performance. To overcome these issues, the authors propose a novel approach combining SMOTE to handle data imbalance, Principal Component Analysis (PCA) for dimensionality reduction, and Ensemble Feature Selection (EFS) for selecting critical features. The evaluation demonstrates significant improvements: the AdaBoost classifier with PCA and SMOTE achieves an Area Under the Receiver Operating Characteristic curve (AUROC) of 92%, while the AdaBoost classifier with EFS and SMOTE achieves 81.83% accuracy, 81.83% precision, 100% recall, and 90.01% F1 Score.

This paper's authors [3] propose a novel class imbalance processing technology called Bayesian Model-based Clustering with Dissimilarities (BMCD), which adapts the SMOTE for large-scale multiclass datasets. The three selected common classification algorithms, namely Naïve Bayes (NB), Random Forest (RF), and Multilayer Perceptron (MLP), were used to compare the performance of classifying balanced and imbalanced combined datasets. The goal is to enhance the detection rate of minority classes while maintaining efficiency. The study combines five individual datasets from the CICIDS2017 to create a comprehensive multiclass dataset containing various types of attacks. To evaluate the effectiveness of the BMCD algorithm, multiple machine learning algorithms are applied to the dataset, paper authors both with and without BMCD. The experimental results demonstrate that BMCD significantly improves intrusion detection performance, particularly for minority classes, and outperforms existing state-of-the-art methods. This highlights BMCD as an effective solution for handling imbalanced datasets in IDS.

The paper's authors [4] explore various machine learning techniques to improve intrusion detection, addressing the challenge of imbalanced datasets, which negatively impacts classification and anomaly detection performance. Two benchmark datasets, NSL-KDD and CICIDS2017, are used to evaluate the proposed methods. The Random Forest classifier is employed for feature selection to identify the most relevant features. The study implements a range of supervised (e.g., K-Nearest Neighbors, Naive Bayes, Random Forest, Decision Trees, Logistic Regression) and unsupervised (e.g., K-Means, Isolation Forest, Local Outlier Factor) learning algorithms. The results show that Random Forest outperforms other supervised learning methods, while K-Means excels among unsupervised techniques. These findings underscore the effectiveness of machine learning in improving intrusion detection, particularly when addressing dataset imbalance and selecting optimal features.

This paper's authors [5] address the challenge of detecting minority intrusion classes in the CICIDS2017 dataset, where traditional learning algorithms tend to favor majority classes. To improve the detection rate for intrusions, the authors explore sampling techniques, including Random Under-Sampling (RUS), SMOTE, and a combination of RUS and SMOTE. After applying these techniques, intrusion detection is performed, and the Decision Tree results are evaluated using accuracy and True Positive Rate (TPR) as metrics. The findings reveal that RUS delivers the best overall detection performance. Additionally, detection results improved for 12 out of 15 classes, including some hard-to-detect minority classes. This demonstrates the effectiveness of sampling techniques, particularly RUS, in enhancing intrusion detection for imbalanced datasets. The paper employed random under-sampling (RUS) on the majority Benign class to mitigate class imbalance and enhance detection performance. Using C4.5 (selected for its superior performance among single classifiers), the best results were achieved by reducing the Benign (no-attack) instances to 30%–90% of their original size. This approach maintained a high average true positive rate

(TPR) of 0.9985, comparable to the full dataset, while improving detection for 12 attack classes, including four previously underrepresented (minority) intrusion types.

This paper's authors [29] leverage machine learning to develop an effective IDS using the CICIDS2017 dataset, addressing two key challenges: high-dimensional features and class imbalance. Authors employ both Auto-Encoders (AE) and Principal Component Analysis (PCA) to dramatically reduce features from 81 to 10 dimensions while maintaining 99.6% classification accuracy across both binary and multiclass scenarios. Authors work introduces CombinedMc, a novel evaluation metric that comprehensively assesses performance by incorporating detection rate, false alarm rate, accuracy, and class distribution. Additionally, authors propose a Uniform Distribution-Based Balancing (UDBB) technique to handle dataset imbalance. The system demonstrates strong performance with various classifiers, including Random Forest (RF), Bayesian Network (BN), Linear Discriminant Analysis (LDA), and Quadratic Discriminant Analysis (QDA), offering an efficient and practical solution for modern intrusion detection that balances computational efficiency with high accuracy.

| Author                                | Method                   | Classes    | ML Model            | Acc (%) | Precision | Recall  | F1(%)   | AUC (%) |
|---------------------------------------|--------------------------|------------|---------------------|---------|-----------|---------|---------|---------|
| Arif Yulianto et al ( 2019) [2]       | SMOTE                    | Binary     | AdaBoost            | 81.83   | 81.83     | 100     | 90.01   | 92      |
| Abdulrahman &Ibrahem et al (2020) [3] | BMCD                     | Multiclass | RF                  | 99.32   |           |         | 98.38   | 98.27   |
|                                       |                          |            | MPL                 | 94.82   |           |         | 80.63   | 94.47   |
|                                       |                          |            | NB                  | 75.35   |           |         | 90.82   | 94.10   |
| Varanasi & Razia et al (2020) [4]     | Near Miss Under sampling | Binary     | Logistic Regression | 82.30   | 54.08     | 65.41   | 59.21   | 89.72   |
|                                       |                          |            | Decision Tree       | 89.15   | 65.48     | 94.72   | 77.43   | 91.06   |
|                                       |                          |            | Random Forest       | 93.77   | 84.14     | 84.15   | 84.14   | 98.61   |
|                                       |                          |            | Gaussian NB         | 69.66   | 31.70     | 47.19   | 37.92   | 76.61   |
|                                       |                          |            | K-Nearest Neighbors | 90.68   | 68.23     | 98.40   | 80.58   | 95.04   |
|                                       |                          |            | KMeans              | 79.00   | [84,46]   | [91,31] | [88,37] |         |
|                                       |                          |            | Isolation Forest    | 79.00   | [45,83]   | [23,93] | [30,88] |         |
| Yan-Bing Ho et al (2021) [5]          | RUS                      | Multiclass | C4.5                | 99.85   |           |         |         |         |
|                                       | SMOTE                    |            |                     | 99.18   |           |         |         |         |
|                                       | RUS+ SMOTE               |            |                     | 99.34   |           |         |         |         |
| Razan Abdulhammed et al (2019) [29]   | UDBB                     | Multiclass | RF                  | 98.8    |           |         | 98.8    |         |
|                                       |                          |            | NB                  | 97.6    |           |         | 97.7    |         |
|                                       |                          |            | LDA                 | 95.7    |           |         | 95.7    |         |
|                                       |                          |            | QDA                 | 98.9    |           |         | 99.0    |         |

**Table 2.1:** Summary of Related Works with CICIDS2017 dataset.

### 3. Data

The **CICIDS2017** (Canadian Institute for Cybersecurity Intrusion Detection System 2017) dataset [6] is a widely used benchmark dataset for network intrusion detection and cybersecurity research. It was created by the Canadian Institute for Cybersecurity (CIC) to provide a comprehensive and realistic dataset for evaluating Intrusion Detection Systems (IDS). The dataset captures real-world network traffic, including normal user activities and various types of cyberattacks.

The dataset is a recording of five days of Benign (normal traffic) and attack traffic. Monday, Tuesday, Wednesday, Thursday and Friday are sub datasets. All these sub datasets are combined to form 78 feature columns and one label column (Benign vs attack name). There are 15 different types of attack traffic. Total amount of observation is 2,830,744.

| Day       | Observations | Traffic Types in Label Column                                       |
|-----------|--------------|---------------------------------------------------------------------|
| Monday    | 529919       | Benign                                                              |
| Tuesday   | 445910       | Benign, Brute Force, FTP-Patator, SSH-Patator                       |
| Wednesday | 692704       | Benign, DoS / DDoS, slowloris, Slowhttptest, Hulk, GoldenEye        |
| Thursday  | 458970       | Benign, Web Attack (Brute Force, XSS, Sql Injection) & Infiltration |
| Friday    | 703248       | Benign, Botnet ARES, Port Scan, DDoS LOIT                           |

**Table 3.1:** Days with Traffic Types

On Monday, a total of 529,919 observations were recorded, consisting entirely of Benign traffic. This day serves as a baseline for normal network behavior, with no malicious activity present.

On Tuesday, the dataset includes 445,910 observations, capturing a mix of Benign traffic alongside Brute Force, FTP-Patator, and SSH-Patator attacks. These represent credential-guessing attacks targeting authentication services like FTP and SSH, simulating unauthorized login attempts.

Wednesday sees a significant increase in traffic with 692,704 observations, comprising Benign traffic as well as a variety of Denial-of-Service (DoS) and Distributed DoS (DDoS) attacks such as slowloris, Slowhttptest, HULK, and GoldenEye. This day highlights both high- and low-rate DoS attack techniques aimed at overwhelming network resources.

On Thursday, the dataset logs 458,970 observations, containing Benign traffic and various web-based attacks, including Brute Force on web forms, Cross-Site Scripting (XSS), and SQL Injection, along with Infiltration attacks. This represents more targeted application-layer attacks and lateral movement within the network.

Finally, Friday has the highest volume with 703,248 observations, capturing Benign traffic, Botnet ARES activity, Port Scanning, and DDoS attacks using the LOIT tool. This day simulates advanced persistent threats and network reconnaissance behavior typically seen in large-scale coordinated attacks.

| Traffic Type             | Observations | %       |
|--------------------------|--------------|---------|
| Benign                   | 2273097      | 80.3004 |
| DoS Hulk                 | 231073       | 8.1630  |
| PortScan                 | 158930       | 5.6144  |
| DDoS                     | 128027       | 4.5227  |
| DoS GoldenEye            | 10293        | 0.3636  |
| FTP-Patator              | 7938         | 0.2804  |
| SSH-Patator              | 5897         | 0.2083  |
| DoS slowloris            | 5796         | 0.2048  |
| DoS Slowhttptest         | 5499         | 0.1943  |
| Bot                      | 1966         | 0.0695  |
| Web Attack Brute Force   | 1507         | 0.0532  |
| Web Attack XSS           | 652          | 0.0230  |
| Infiltration             | 36           | 0.0013  |
| Web Attack Sql Injection | 21           | 0.0007  |
| Heartbleed               | 11           | 0.0004  |

**Table 3.2:** Traffic Types with Observations and Percentage [5]

### 3.1 Network Traffic Types

The network traffic types contained in the dataset are categorized and described in the following section [15].

#### Benign

Benign refers to normal, legitimate network traffic that is not associated with any malicious or suspicious activity. Benign traffic = normal behavior = not an attack.

#### DoS Hulk

HULK (HTTP Unbearable Load King) is a Denial of Service (DoS) attack tool that floods a web server with a large number of uniquely crafted HTTP requests. It is designed to evade caching and flood the server with continuous traffic, exhausting its resources and causing it to crash or become unresponsive.

#### PortScan

Port scanning is a technique used by attackers to find open ports on a host. By scanning different ports, they can determine which services are running and identify potential vulnerabilities.

#### DDoS (Distributed Denial of Service)

DDoS attacks involve multiple compromised systems (usually part of a botnet) that flood a target system or network with traffic, rendering it inaccessible to legitimate users. These attacks are difficult to stop because they originate from many sources.

## **DoS GoldenEye**

GoldenEye is a type of DoS attack that bombards a server with HTTP POST and GET requests. The goal is to overload the server's handling capacity by keeping many connections open, forcing it to slow down or crash.

## **FTP-Patator**

FTP-Patator is a brute-force attack tool used to guess username and password combinations for FTP services. It systematically tries multiple credentials in order to gain unauthorized access to an FTP server.

## **SSH-Patator**

SSH-Patator is similar to FTP-Patator but targets the Secure Shell (SSH) protocol. It performs brute-force attacks to gain access to remote systems via SSH by guessing login credentials.

## **DoS slowloris**

Slowloris is a low-bandwidth DoS attack that targets web servers by opening many connections and sending partial HTTP headers very slowly. This keeps the connections alive and uses up server resources, preventing legitimate users from connecting.

## **DoS Slowhttptest**

An advanced slow-rate DoS attack tool that tests a server's vulnerability to attacks like Slowloris, Slow POST, or range-based slow reads.

## **Bot**

A bot refers to an infected computer that is under the control of an attacker. Bots are often used in coordinated attacks like DDoS or for spreading malware. They can operate autonomously or under the command of a botmaster.

## **Web Attack – Brute Force**

This type of attack involves repeatedly trying different combinations of usernames and passwords on a web login page. It aims to gain unauthorized access to a website or system by guessing valid credentials.

## **Web Attack – XSS (Cross-Site Scripting)**

XSS attacks involve injecting malicious scripts into web applications. When users visit the affected web page, the script executes in their browsers, potentially stealing cookies, session tokens, or redirecting them to malicious sites.

## **Infiltration**

Infiltration refers to unauthorized access into a network, usually by bypassing security mechanisms. It is often the first step in a larger cyberattack where the attacker establishes a foothold within the system.

## **Web Attack – SQL Injection**

SQL Injection occurs when an attacker inserts malicious SQL statements into input fields of a web application. If the input is not properly sanitized, the attacker can manipulate the database to access or alter sensitive data.

## Heartbleed

Heartbleed is a vulnerability in the OpenSSL cryptographic library. It allows attackers to read portions of server memory, potentially exposing private keys, passwords, and other confidential data. It affects services that use SSL/TLS for secure communication.

## 3.2 Explanation of Network Traffic Features

The dataset comprises 78 feature columns, each representing distinct network traffic characteristics. The following section provides detailed descriptions of these features and their significance for intrusion detection analysis.

### Basic Flow Features

These features describe the general properties of a network flow (a sequence of packets between two endpoints).

| <i>Feature</i>                  | <i>Description</i>                                                      |
|---------------------------------|-------------------------------------------------------------------------|
| Destination Port                | Port number of the receiving device (e.g., 80 for HTTP, 443 for HTTPS). |
| Flow Duration                   | Total time taken for the network flow (in microseconds).                |
| Total Fwd Packets               | Number of packets sent from the source to destination.                  |
| Total Backward Packets          | Number of packets sent from the destination to source.                  |
| Total Length of Fwd/Bwd Packets | Total size of forward/backward packets.                                 |
| Fwd Packets/s / Bwd Packets/s   | Packet sending rate in both directions.                                 |

### Packet Size Features

These features describe the size of packets sent in both directions (forward and backward).

| <i>Feature</i>              | <i>Description</i>                                            |
|-----------------------------|---------------------------------------------------------------|
| Total Length of Fwd Packets | Total bytes sent in the forward direction.                    |
| Total Length of Bwd Packets | Total bytes received in the backward direction.               |
| Fwd Packet Length Max       | Maximum packet size in the forward direction.                 |
| Fwd Packet Length Min       | Minimum packet size in the forward direction.                 |
| Fwd Packet Length Mean      | Average packet size in the forward direction.                 |
| Fwd Packet Length Std       | Standard deviation of packet sizes in the forward direction.  |
| Bwd Packet Length Max       | Maximum packet size in the backward direction.                |
| Bwd Packet Length Min       | Minimum packet size in the backward direction.                |
| Bwd Packet Length Mean      | Average packet size in the backward direction.                |
| Bwd Packet Length Std       | Standard deviation of packet sizes in the backward direction. |

### Flow Rate Features

These features describe the speed of network traffic.

| <i>Feature</i> | <i>Description</i>                               |
|----------------|--------------------------------------------------|
| Flow Bytes/s   | Total number of bytes per second for the flow.   |
| Flow Packets/s | Total number of packets per second for the flow. |

## Inter-Arrival Time (IAT) Features

IAT refers to the time gap between two consecutive packets in the flow.

| <i>Feature</i> | <i>Description</i>                                   |
|----------------|------------------------------------------------------|
| Flow IAT Mean  | Average time difference between packets in the flow. |
| Flow IAT Std   | Standard deviation of IAT values in the flow.        |
| Flow IAT Max   | Maximum inter-arrival time between packets.          |
| Flow IAT Min   | Minimum inter-arrival time between packets.          |

Similar features exist for **forward (Fwd)** and **backward (Bwd)** traffic:

Fwd IAT Total, Fwd IAT Mean, Fwd IAT Std, Fwd IAT Max, Fwd IAT Min (for packets sent from source to destination).

Bwd IAT Total, Bwd IAT Mean, Bwd IAT Std, Bwd IAT Max, Bwd IAT Min (for packets sent from destination to source).

## TCP Flag Features

TCP flags indicate the status of the connection and are crucial for identifying attacks (e.g., DoS, port scanning).

| <i>Feature</i> | <i>Description</i>                                             |
|----------------|----------------------------------------------------------------|
| FIN Flag Count | Number of FIN packets (used to terminate connections).         |
| SYN Flag Count | Number of SYN packets (used to initiate connections).          |
| RST Flag Count | Number of RST packets (used to reset connections).             |
| PSH Flag Count | Number of PSH packets (used to push data immediately).         |
| ACK Flag Count | Number of ACK packets (used to acknowledge received data).     |
| URG Flag Count | Number of URG packets (used for urgent data).                  |
| CWE Flag Count | Number of Congestion Window Reduced (CWE) packets.             |
| ECE Flag Count | Number of Explicit Congestion Notification Echo (ECE) packets. |

## Header & Protocol Features

These features describe TCP/IP headers and protocol-level information.

| <i>Feature</i>      | <i>Description</i>                              |
|---------------------|-------------------------------------------------|
| Fwd Header Length   | Total header size of packets sent forward.      |
| Bwd Header Length   | Total header size of packets received backward. |
| Down/Up Ratio       | Ratio of backward to forward traffic.           |
| Average Packet Size | Mean size of all packets in the flow.           |

## Bulk Transfer Features

These features describe how packets are sent in bulk within a network flow.

| <i>Feature</i>       | <i>Description</i>                                    |
|----------------------|-------------------------------------------------------|
| Fwd Avg Bytes/Bulk   | Average number of bytes in a bulk forward transfer.   |
| Fwd Avg Packets/Bulk | Average number of packets in a bulk forward transfer. |

|                      |                                                        |
|----------------------|--------------------------------------------------------|
| Fwd Avg Bulk Rate    | Rate of bulk transfers in the forward direction.       |
| Bwd Avg Bytes/Bulk   | Average number of bytes in a bulk backward transfer.   |
| Bwd Avg Packets/Bulk | Average number of packets in a bulk backward transfer. |
| Bwd Avg Bulk Rate    | Rate of bulk transfers in the backward direction.      |

### Subflow Features

A subflow refers to a subset of packets within the main flow.

| <i>Feature</i>      | <i>Description</i>                     |
|---------------------|----------------------------------------|
| Subflow Fwd Packets | Number of packets in forward subflow.  |
| Subflow Fwd Bytes   | Number of bytes in forward subflow.    |
| Subflow Bwd Packets | Number of packets in backward subflow. |
| Subflow Bwd Bytes   | Number of bytes in backward subflow.   |

### Window Size Features

These features describe TCP window size settings.

| <i>Feature</i>          | <i>Description</i>                            |
|-------------------------|-----------------------------------------------|
| Init_Win_bytes_forward  | Initial TCP window size for forward packets.  |
| Init_Win_bytes_backward | Initial TCP window size for backward packets. |

### Active & Idle Time Features

These features describe how long a flow remains active before going idle.

| <i>Feature</i> | <i>Description</i>                                            |
|----------------|---------------------------------------------------------------|
| Active Mean    | Average duration of active time within a flow.                |
| Active Std     | Standard deviation of active time.                            |
| Active Max     | Maximum duration of active time.                              |
| Active Min     | Minimum duration of active time.                              |
| Idle Mean      | Average duration of idle time before the next packet is sent. |
| Idle Std       | Standard deviation of idle time.                              |
| Idle Max       | Maximum idle time.                                            |
| Idle Min       | Minimum idle time.                                            |

### Packet Size Statistics

These features describe the size and distribution of packets in the flow

| <i>Feature</i>         | <i>Description</i>                      |
|------------------------|-----------------------------------------|
| Packet Length Mean     | Average size of all packets in the flow |
| Packet Length Std      | Standard deviation of packet sizes      |
| Packet Length Variance | Variability in packet size              |
| Max Packet Length      | Largest packet size in the flow         |

### Directional Segment Metrics

These refer to data segments flowing in forward or backward directions.

| <i>Feature</i>       | <i>Description</i>                                        |
|----------------------|-----------------------------------------------------------|
| Avg Fwd Segment Size | Average size of data segments from source to destination. |
| Avg Bwd Segment Size | Average size of data segments from destination to source. |

### Application/Data-Level Features

These reflect actual data content in the flow.

| <i>Feature</i>   | <i>Description</i>                                                         |
|------------------|----------------------------------------------------------------------------|
| act_data_pkt_fwd | Number of packets carrying application-layer data in the forward direction |

### Target Variable (Label)

| <i>Feature</i> | <i>Description</i>                                                                                |
|----------------|---------------------------------------------------------------------------------------------------|
| Label          | Indicates whether the traffic is <b>Benign</b> or an <b>attack</b> (e.g., DDoS, Port Scan, etc.). |

| Bwd Avg F | Bwd Avg B | Subflow Fv | Subflow F | Subflow B | Subflow B | Init_Win_t | Init_Win_ | act_data_ | min_seg_ | Active Me | Active Std | Active Ma | Active Mir | Idle Mean | Idle Std | Idle Max | Idle Min | Label    |
|-----------|-----------|------------|-----------|-----------|-----------|------------|-----------|-----------|----------|-----------|------------|-----------|------------|-----------|----------|----------|----------|----------|
| 0         | 0         | 2          | 12        | 0         | 0         | 33         | -1        | 1         | 20       | 0         | 0          | 0         | 0          | 0         | 0        | 0        | 0        | 0 BENIGN |
| 0         | 0         | 1          | 6         | 1         | 6         | 29         | 256       | 0         | 20       | 0         | 0          | 0         | 0          | 0         | 0        | 0        | 0        | 0 BENIGN |
| 0         | 0         | 1          | 6         | 1         | 6         | 29         | 256       | 0         | 20       | 0         | 0          | 0         | 0          | 0         | 0        | 0        | 0        | 0 BENIGN |
| 0         | 0         | 1          | 6         | 1         | 6         | 31         | 329       | 0         | 20       | 0         | 0          | 0         | 0          | 0         | 0        | 0        | 0        | 0 BENIGN |
| 0         | 0         | 2          | 12        | 0         | 0         | 32         | -1        | 1         | 20       | 0         | 0          | 0         | 0          | 0         | 0        | 0        | 0        | 0 BENIGN |
| 0         | 0         | 2          | 12        | 0         | 0         | 32         | -1        | 1         | 20       | 0         | 0          | 0         | 0          | 0         | 0        | 0        | 0        | 0 BENIGN |
| 0         | 0         | 2          | 12        | 0         | 0         | 32         | -1        | 1         | 20       | 0         | 0          | 0         | 0          | 0         | 0        | 0        | 0        | 0 BENIGN |
| 0         | 0         | 1          | 6         | 1         | 6         | 32         | 256       | 0         | 20       | 0         | 0          | 0         | 0          | 0         | 0        | 0        | 0        | 0 BENIGN |
| 0         | 0         | 2          | 12        | 0         | 0         | 32         | -1        | 1         | 20       | 0         | 0          | 0         | 0          | 0         | 0        | 0        | 0        | 0 BENIGN |
| 0         | 0         | 2          | 37        | 0         | 0         | 946        | -1        | 1         | 20       | 0         | 0          | 0         | 0          | 0         | 0        | 0        | 0        | 0 BENIGN |

**Figure 3.1:** Part of Dataset

## 4. Theory

This section provides a detailed explanation and expands on all the theoretical groundwork necessary to implement the majority of the techniques discussed in this study. Logistic Regression, K-Nearest Neighbors (KNN), Naive Bayes, Random Forest, Deep Neural Network (DNN) in supervised learning model and K – Means, Isolation Forest in unsupervised learning model are explained.

### 4.1 Supervised Learning

Supervised learning is a foundational machine learning technique where models are trained on labeled datasets, meaning each input (features) is paired with a known output (label or target value). The objective is to derive a mapping function capable of generalizing to novel input data with high predictive accuracy. This approach is widely used in tasks where historical data with clear outcomes is available. Supervised learning is broadly categorized into *classification* and *regression*, depending on whether the output is a discrete category or a continuous numerical value [26]. The classification task aims to assign categorical labels to instances based on their characteristic features. A common example is *binary classification*, which distinguishes between two classes. Supervised learning algorithms which are used in this study, *Naive Bayes*, *Random Forests*, *K-Nearest Neighbors*, *Deep Neural Network (DNN)* and *Logistic Regression* are classification algorithms. Regression predicts continuous numerical values. For instance, it can forecast stock prices, estimate house values, or predict temperature trends based on historical data. Unlike classification, regression outputs are real numbers, and models are evaluated using metrics [26].

### 4.2 Unsupervised Learning

Unsupervised learning constitutes a fundamental paradigm within machine learning wherein computational algorithms process and analyze datasets that lack predefined labels or target variables. Unlike supervised learning techniques that necessitate precisely labeled datasets to inform the learning algorithm, this approach operates under different constraints. The defining characteristic of unsupervised techniques lies in their ability to autonomously examine the intrinsic structure of data, revealing hidden patterns, meaningful groupings, and complex relationships that might not be immediately apparent to human observers. This renders unsupervised methods ideal for situations with insufficient, costly, or nonexistent labeled data [26].

The field encompasses several distinct but interrelated methodologies, each designed to address specific aspects of data exploration and knowledge discovery. Clustering algorithms represent one of the most prominent categories, employing various mathematical approaches to partition datasets into meaningful subgroups based on similarity measures. *K-Means* clustering operates by iteratively assigning data points to centroids, while *Hierarchical Clustering* builds nested clusters through either agglomerative (bottom-up) or divisive (top-down) strategies. Density-based methods like *Density-Based Spatial Clustering of Applications with Noise* (DBSCAN) offer particular advantages for datasets with irregular cluster shapes, and probabilistic approaches such as *Gaussian Mixture Models* provide a flexible framework for modeling cluster distributions [26].

Anomaly detection represents a crucial application area where unsupervised learning excels, particularly in identifying rare events or outliers that may indicate significant occurrences. Techniques such as *Isolation Forest*, which isolates anomalies rather than profiling normal instances, and *One-Class SVM*, which learns

a tight boundary around normal data points, have become standard tools for fraud detection, network security, and quality control applications.

These methods are especially valuable because they can operate without labeled examples of anomalies, which are often extremely rare in real-world scenarios. The unsupervised methods were chosen to complement the supervised analysis: K-Means provides a baseline for clustering-based anomaly detection, while Isolation Forest is specifically designed for outlier identification, making it well-suited for attack scenarios where anomalies (attacks) are rare compared to normal data.

## 4.3 Sampling

Sampling is a crucial technique in data science and machine learning, used to select a representative subset of data from a larger dataset. This process helps reduce computational costs, mitigate overfitting, and improve model generalization. However, when dealing with imbalanced datasets where one class significantly outnumbers another, standard sampling may not suffice. In such cases, specialized techniques like *oversampling*, *undersampling*, and *hybrid approaches* are employed to ensure that machine learning models learn effectively without bias toward the dominant class. These methods help balance class distributions, allowing classifiers to make more accurate predictions for minority classes, which is particularly important in applications like fraud detection, medical diagnosis, and anomaly detection [5].

### Oversampling Techniques

Oversampling, meaning one class has significantly fewer instances than the other. Its principal objective is to augment the representation of the minority class, preventing the model from developing a predisposition toward the predominant class. The most basic method, *Random Oversampling* (ROS), creates additional instances by replicating existing minority class entries. In response to this limitation, more complex analytical methods have been engineered. The *SMOTE* generates new synthetic samples by interpolating between neighboring minority class instances, thereby introducing diversity into the dataset. [5].

### Undersampling Techniques

Undersampling reduces the quantity of majority class examples, creating a more equitable distribution between classes. The most straightforward approach, *Random Undersampling* (RUS), eliminates the majority of instances arbitrarily, though this may inadvertently discard potentially valuable data points. *Near Miss Undersampling* selects the majority samples that are closest to minority instances, ensuring that critical decision boundaries are retained.[5].

### Hybrid Approaches

Hybrid methods combine oversampling and undersampling to leverage the strengths of both while minimizing their weaknesses. For example:

- SMOTE + TomekLinks first generates synthetic minority samples and then removes noisy or overlapping majority instances, resulting in a cleaner and more balanced dataset.

These hybrid approaches are particularly useful in moderately sized datasets where neither oversampling nor undersampling alone provides optimal results. The choice between techniques depends on factors such as dataset size, class imbalance severity, and computational constraints. [5].

## 4.4 Synthetic Minority Oversampling Technique (SMOTE)

Synthetic Minority Oversampling Technique (SMOTE) is a popular oversampling method used to address class imbalance in datasets. SMOTE algorithm was proposed in the Journal of Artificial Intelligence for the first time by **Chawla in 2002** [16]. Class imbalance occurs when one class (the minority class) has significantly fewer samples than the other class (the majority class). This imbalance can lead to biased machine learning models that perform poorly on the minority class. SMOTE helps balance the dataset by generating synthetic samples for the minority class, improving the model's ability to learn and generalize [5].

### How SMOTE Works [17]

#### Identify Minority Class Samples:

SMOTE focuses on the **minority class** in an imbalanced dataset.

#### Select k-Nearest Neighbors:

For each instance in the minority class, SMOTE finds its **k-nearest neighbors** within the same class using a distance metric.

#### Synthesize New Samples:

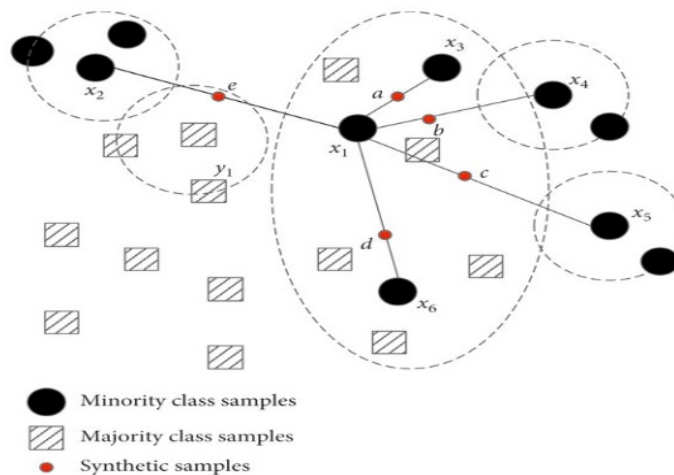
SMOTE randomly selects one of the k-nearest neighbors. It generates a synthetic sample by interpolating between the chosen sample and its neighbor:

$$\text{Synthetic Sample} = x_i + \lambda \cdot (x_j - x_i)$$

- $x_i$  : Original minority sample.
- $x_j$  : Neighbor minority sample.
- $\lambda$ : A random number between 0 and 1.

#### Add Synthetic Samples to Dataset:

These synthetic samples are added to the dataset, balancing the minority class with the majority class.



**Figure 4.1:** The schematic of SMOTE algorithm [52].

SMOTE operates by identifying neighboring samples within the minority class in the feature space and generating synthetic samples along the line segments connecting them. The process begins by randomly selecting a minority class sample and then identifying its  $k$  nearest neighbors (default  $k = 5$ ) [18,16]. In this study, use  $k = 5$ . A neighbor is randomly chosen, and a new synthetic sample is created at an intermediate point between the two in the feature space. However, a key limitation of SMOTE is that it may generate synthetic samples within regions predominantly occupied by the majority class. This happens when the selected minority sample and its neighbors lie near the decision boundary or within majority-class-dominated areas. As a result, these synthetic samples can introduce noise, blur class boundaries, and degrade classification performance. In this study, synthetic data is generated comprising two distinct classes within the  $[0.0, 1.0]$  range, representing 'Attack' and 'Benign' categories, respectively [16]. The 'Attack' samples are increased to match their original size (doubled) through synthetic generation.

To mitigate some of the issues, various SMOTE variants have been proposed. These variants aim to balance effective oversampling of the minority class while minimizing noise generation, thereby reducing overfitting and improving prediction accuracy [18]. These variants are designed to generate synthetic samples in a more controlled manner, reducing sensitivity to outliers while improving the reliability of the resulting models.

**Borderline-SMOTE** [20] is an advanced oversampling technique that improves upon the basic SMOTE algorithm by strategically targeting minority class instances near the decision boundary. Unlike SMOTE, which generates synthetic samples uniformly across the minority class, Borderline-SMOTE focuses only on "dangerous" minority samples those located in regions where the majority class dominates their local neighborhood. The algorithm begins by analyzing each minority instance and its  $k$  nearest neighbors to determine whether it lies in a borderline area. If a minority sample has a significant proportion of majority-class neighbors, it is classified as "dangerous" and selected for oversampling. In contrast, minority samples surrounded mostly by other minority instances are considered "safe" and ignored, while those entirely surrounded by majority samples are treated as noise and discarded. By concentrating synthetic sample generation on these critical borderline cases, Borderline-SMOTE strengthens the minority class representation in ambiguous regions without introducing excessive noise. This targeted approach enhances classifier performance by refining decision boundaries where misclassification errors are most likely to occur, making it particularly effective for imbalanced datasets with complex class overlaps.

**ADASYN Adaptive Synthetic Sampling** [21] is an intelligent oversampling approach that enhances the standard SMOTE algorithm by adaptively generating synthetic samples according to the local distribution and learning difficulty of minority class instances. The core innovation of ADASYN lies in its dynamic weighting mechanism, which assigns higher importance to minority samples located in complex decision regions where class overlap occurs. By analyzing the neighborhood composition of each minority instance, ADASYN identifies harder-to-classify examples that are surrounded predominantly by majority class samples and strategically generates more synthetic data points in these challenging regions. This adaptive approach not only improves the classifier's ability to learn meaningful decision boundaries in imbalanced datasets but also automatically reduces the emphasis on easier-to-classify minority instances and potential outliers. The method first calculates a density distribution to determine the optimal number of synthetic samples needed for each minority example, then creates interpolated points that effectively strengthen the representation of the most vulnerable areas of the minority class. Compared to uniform oversampling techniques, ADASYN produces more sophisticated synthetic datasets that better reflect the true underlying data distribution, leading to more robust machine learning models, particularly for classification tasks where the minority class exhibits significant overlap with the majority class in the feature space.

**SMOTE-TomekLinks** [22] is a hybrid approach that combines the synthetic oversampling of SMOTE with the noise-filtering capability of Tomek Links. The method first applies SMOTE to generate synthetic minority class samples, balancing the class distribution. Subsequently, Tomek Links, a data-cleaning technique, identifies and removes overlapping or ambiguous instances near the decision boundary that may represent noise or outliers. By eliminating these problematic pairs (where each instance is the nearest neighbor of the other but belongs to opposing classes), the algorithm refines the dataset, sharpening class separability. This two-step process not only mitigates class imbalance but also enhances the quality of the training data by reducing misleading overlaps. As a result, classifiers trained on the refined dataset achieve improved generalization and performance, particularly in scenarios where noisy or borderline instances could otherwise degrade model accuracy. The integration of oversampling and intelligent undersampling makes SMOTE-TomekLinks a robust solution for handling imbalanced datasets with complex class boundaries.

## 4.5 Deep Neural Network (DNN)

A Deep Neural Network (DNN) is a type of Artificial Neural Network (ANN) that features multiple layers of interconnected neurons between the input and output layers. DNNs are widely used in modern machine learning tasks due to their capacity to learn hierarchical representations and model complex relationships in data. A deep neural network (DNN) is architecturally composed of three primary components: an input layer, several hidden layers, and an output layer. Within each layer, neurons compute a weighted sum of their inputs and subsequently apply a nonlinear activation function. Mathematically, the operation performed by a neuron in layer “ $l$ ” can be expressed as

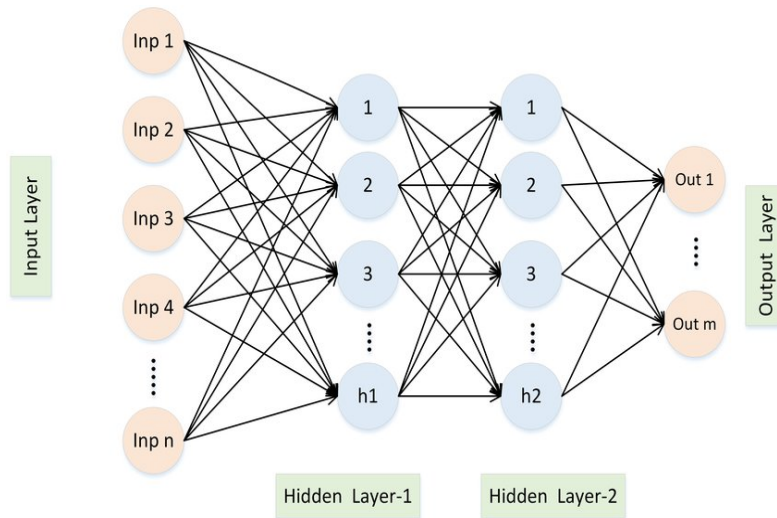
$$z_{(l)} = W_{(l)} * a_{(l-1)} + b_{(l)}$$

$W_{(l)}$ : The weight matrix.

$a_{(l-1)}$ : The activation from the previous layer.

$b_{(l)}$ : The bias vector.

$z_{(l)}$ : The output of the neuron.



**Figure 4.2:** General Architecture for a Deep Neural Network with Two Hidden Layers. [53]

The most common layer type in DNNs is the dense or fully connected layer, where each neuron in a layer receives input from every neuron in the previous layer. This dense connectivity allows for the learning of complex patterns but also increases the computational load and the risk of overfitting. The output of a dense layer is often passed through an *activation function*, which introduces non-linearity into the model. The *Rectified Linear Unit* (ReLU) is a popular activation function defined as  $\text{ReLU}(x) = \max(0, x)$ . It is simple and efficient, and it helps mitigate the vanishing gradient problem, which can occur with other activation functions like sigmoid or tanh. However, ReLU can lead to dead neurons, which are neurons that stop activating during training [30, 31, 32].

Another widely used activation function is the *sigmoid function*, especially in binary classification tasks. It is defined as,

$$\sigma(x) = \frac{1}{(1 + e^{-x})}$$

which maps input values to a range between 0 and 1. Despite its usefulness, the sigmoid function can saturate for large positive or negative input values, causing the gradients to become very small during backpropagation and leading to slow training. Therefore, in deeper networks, ReLU is often preferred in the hidden layers.

*Dropout* layers are often used to address the issue of overfitting in deep networks. Dropout is a regularization technique where a random subset of neurons is ignored during training. This means that during each training iteration, certain neurons are "dropped out" with a specified probability " $p$ ", and they do not contribute to the forward pass or backpropagation. The operation can be described as  $a\_drop = a * r$ , where  $r$  is a vector of Bernoulli random variables with probability  $p$  of being 1. This technique forces the network to learn redundant representations and prevents neurons from co-adapting too much, thereby improving generalization [30, 31, 32].

*Batch Normalization* is another important technique used in DNNs to stabilize and accelerate the training process. It works by normalizing the inputs of each layer to have a mean of zero and a variance of one, across a mini-batch of data. For a batch of inputs  $x = \{x_1, x_2, \dots, x_m\}$ , the batch mean  $\mu$  and variance  $\sigma^2$  are computed as,

$$\mu = \frac{1}{m} \sum_{i=1}^m x_i$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2$$

Each input is then normalized as,

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$\epsilon$  : A small constant ( $10^{-5}$ ) for numerical stability.

The result is scaled and shifted using *learnable parameters*  $\gamma$  and  $\beta$ :  $y_i = \gamma * \hat{x}_i + \beta$ . This normalization helps to reduce the internal covariate shift and allows the network to use higher learning rates.

Training a DNN involves adjusting its weights and biases to minimize a chosen *loss function*. This is done using the backpropagation algorithm, which efficiently computes the gradient of the loss function with respect to each parameter in the network. The loss function could be *Mean Squared Error* (MSE) for regression tasks, defined as,

$$L_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

*Binary Cross-Entropy* (BCE) for binary classification, defined as,

$$L_{\text{BCE}} = - \frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

The *gradients* are computed in the backward pass using the chain rule. For a weight matrix  $W_{(l)}$ , the gradient is,

$$\frac{\partial L}{\partial W_{(l)}} = \delta_{(l)} * (a_{(l-1)})^T$$

where  $\delta_{(l)}$  is the error at layer  $l$ . The error is recursively defined as,

$$\delta_{(l)} = (W_{(l+1)})^T * \delta_{(l+1)} \circ f'(z_{(l)})$$

where  $\circ$  denotes element-wise multiplication and  $f'$  is the derivative of the activation function [30,31,32].

Once the gradients are known, the network's parameters are updated using an optimization algorithm like *Stochastic Gradient Descent* (SGD). The update rule is,

$$W_{(l)} = W_{(l)} - \eta * \frac{\partial L}{\partial W_{(l)}}$$

where  $\eta$  is the *learning rate*. More advanced optimizers like *Adam* or *RMSprop* use adaptive learning rates and momentum terms to accelerate convergence and improve performance. These optimizers dynamically adjust the learning rate for each parameter based on estimates of first and second moments of the gradients.

## 4.6 K-Nearest Neighbors (KNN)

The k-nearest neighbors (KNN) algorithm is a non-parametric, classification and regression problems. Instead, it classifies new data points based on their proximity to existing labeled data. For classification, the algorithm assigns a class label by conducting a majority vote among the “ $k$ ” closest training samples. A crucial aspect of KNN is measuring the similarity between data points. The algorithm relies on distance metrics to identify the nearest neighbors of a new data point in the set of labeled data [4]. Common distance measures include: Euclidean Distance is the straight-line distance between two points, calculated as the square root of the sum of squared differences. Manhattan Distance is the sum of absolute differences along each axis, suitable for grid-like paths. Minkowski Distance is a versatile distance metric that generalizes both Euclidean and Manhattan distances by introducing a parameter “ $p$ ”, which adjusts the distance calculation based on different geometric interpretations [38]. The Minkowski distance is defined as:

$$\text{Minkowski distance} = (\sum_{i=1}^n |x_i - y_i|^p)^{1/p}$$

$p$ : A parameter that determines the type of distance.

$p = 1$ : Manhattan Distance.

$p = 2$ : Euclidean Distance.

$x_i - y_i$ : difference between the coordinates of  $x$  and  $y$  along the  $i$  th dimension.

$x = (x_1, x_2, \dots, x_n)$ : A vector representing the coordinates of the first point.

$y = (y_1, y_2, \dots, y_n)$ : A vector representing the coordinates of the second point.

## 4.7 Naïve Bayes

Naïve Bayes is a probabilistic classifier rooted in Bayes' Theorem. It computes the class conditional probabilities and prior probabilities to derive the posterior probability. The classifier then predicts the class with the highest posterior probability from a set of possible classes for a given input. The algorithm functions by identifying the class that maximizes the posterior probability. There are several types of Naïve Bayes classifiers, each tailored to different distributions of feature values. The most common variants include: Gaussian Naïve Bayes (GaussianNB), Multinomial Naïve Bayes (MultinomialNB), and Bernoulli Naïve Bayes (BernoulliNB) [4]. In this study, *Gaussian Naïve Bayes (GaussianNB)* is used for classification. Gaussian Naïve Bayes (GaussianNB): This version assumes that features follow a Gaussian (normal) distribution and is used for continuous data. The model is trained by calculating the mean and standard deviation for each class, assuming the likelihood of features is Gaussian [50].

### 1. Bayes' Theorem (Base Formula)

$$P(c_k | x) = \frac{P(c_k) \cdot P(x | c_k)}{P(x)}$$

$P(c_k)$  : Prior probability of class  $c_k$  .

$P(x | c_k)$  : Likelihood of observing  $x$  given class  $c_k$  .

$P(x)$  : Marginal probability.

$x = (x_1, x_2, \dots, x_n)$ : Feature vector representing a single data point with  $n$  features.

$c_k$ :  $k$  th class label

Since  $P(x)$  is the same for all classes,

$$P(c_k | x) \propto P(c_k) \cdot P(x | c_k)$$

### 2. Naïve Assumption (Conditional Independence)

The "Naïve" assumption states that features are independent given the class:

$$P(x | c_k) = \prod_{i=1}^n p(x_i | c_k)$$

Thus, the posterior probability becomes:

$$P(c_k | x) \propto P(c_k) \cdot \prod_{i=1}^n p(x_i | c_k)$$

### 3. Gaussian (Normal) Distribution Assumption

For continuous features, assume each  $x_i$  follows a Gaussian distribution per class  $c_k$

$$p(x_i | c_k) = \frac{1}{\sqrt{2\pi\sigma_{k,i}^2}} \exp\left(-\frac{(x_i - \mu_{k,i})^2}{2\sigma_{k,i}^2}\right)$$

$\mu_{k,i}$  : Mean of feature  $x_i$  for class  $c_k$

$\sigma_{k,i}^2$  : Variance of feature  $x_i$  for class  $c_k$

## 4.8 Random Forest

Random Forest is an ensemble learning method that constructs multiple decision trees during training and combines their predictions to improve accuracy and prevent overfitting. The algorithm begins by creating bootstrap samples from the original dataset - for each of the  $T$  trees [4] in this study, 100 trees, it randomly selects  $N$  observations with replacement, leaving data out-of-bag (OOB) for error estimation. When building each tree, at every node, the algorithm randomly selects a subset of  $m$  features (typically  $m = \sqrt{p}$  for classification,  $p$  refers to the total number of features) and evaluates all possible splits to find the one that maximizes the reduction in Gini impurity ( $\Delta G$ ).

The Gini impurity at node  $m$  is calculated as:

$$G_m = \sum_{k=1}^c p_{mk}(1 - p_{mk})$$

$p_{mk}$  : Probability of class  $k$  at node  $m$ .

This splitting process continues recursively until nodes become pure or other stopping criteria are met. For prediction, the forest aggregates outputs from all trees through majority voting in classification (the mode of all tree predictions) or averaging in regression [37]. The final prediction is the majority vote:

$$\hat{y} = \operatorname{argmax} \sum_{t=1}^T \mathbb{I}(h_t(x) = c)$$

$h_t(x)$  : Prediction of the  $t$  th tree for input  $x$ .

$\mathbb{I}(\cdot)$ : Indicator function (1 if true, 0 otherwise).

$c$  : Number of classes.

The method's strength comes from three key mechanisms: bootstrap aggregating (bagging) that creates diversity among trees, random feature selection that decorrelates the trees, and ensemble averaging that reduces variance. Additionally, the algorithm naturally provides feature importance scores by tracking how much each feature decreases impurity across all splits in all trees, while the out-of-bag (OOB) samples offer an unbiased estimate of the model's generalization error without needing a separate validation set [37]. This combination of randomization and aggregation makes Random Forest robust to noise and capable of handling high-dimensional data while maintaining interpretability through feature importance measures.

## 4.9 Logistic Regression

Logistic regression is a widely used statistical method for binary classification that models the probability of a binary outcome. Logistic regression applies the logistic function (sigmoid function) to transform a linear combination of input features into a probability value between 0 and 1 [51]. The sigmoid function is defined as:

$$\sigma(z) = \frac{1}{(1 + e^{-z})}$$

$$z = \beta_0 + \beta_1 k_1 + \dots + \beta_n k_n$$

$\beta_0$ : Intercept (bias term).

$\beta_0 + \beta_1 + \dots + \beta_n$ : Coefficients for features.

$k = (k_1, \dots, k_n)$ : input features.

The model estimates these coefficients using maximum likelihood estimation (MLE), which optimizes the likelihood of observing the given data under the model [50]. The coefficients ( $\beta$ ) provide insight into the direction and magnitude of each feature's influence on the predicted probability. A positive coefficient indicates that an increase in the feature value raises the probability of the positive class, while a negative coefficient suggests the opposite. Additionally, the model can be extended with regularization techniques (L1 or L2) to prevent overfitting and improve generalization, particularly in high-dimensional datasets [51].

## 4.10 K – Means

K-Means is a fundamental unsupervised learning algorithm that partitions a dataset into K distinct, non-overlapping clusters based on similarity measures.[4] As one of the most widely used clustering techniques in cybersecurity analytics, K-Means operates by iteratively assigning data points to the nearest cluster centroid and recomputing centroids until convergence. The algorithm's objective is to minimize intra-cluster variance while maximizing inter-cluster separation, formally expressed through the optimization function [23]. In this study, use  $k = 2$ .

$$J = \sum_{k=1}^K \sum_{x_i \in c_k} ||x_i - \mu_k||^2$$

$C_k$  : cluster k

$\mu_k$ : centroid of cluster k

$x_i$ : data point

## 4.11 Isolation Forest

Isolation Forest is an efficient unsupervised anomaly detection algorithm that operates on the principle that anomalies are both rare and significantly different from normal instances, making them easier to isolate through random partitioning [4]. Unlike traditional distance or density-based methods, Isolation Forest constructs an ensemble of isolation trees (iTrees) by recursively splitting the feature space using randomly selected features and split values. Each tree continues partitioning until individual instances are isolated, with anomalies requiring fewer splits due to their distinct characteristics.[23] The algorithm calculates an anomaly score for each data point based on the average path length required for isolation, normalized by the expected path length for the given sample size. This score, computed as

$$s(x,n) = 2^{-\frac{E[h(x)]}{c(n)}}$$

$E[h(x)]$  : Average path length across all trees.

$c(n)$ : Normalization factor for dataset size  $n$ .

$h(x)$ : path length.

$c(n)$  is a normalization factor, ranges from 0 to 1, with values closer to 1 indicating higher anomaly likelihood. Practical implementation considerations include setting appropriate parameters such as the number of trees (typically 100-200), subsample size (often 256 instances), and contamination factor (defining the expected proportion of anomalies).

## 4.12 Evaluation Metrics

Evaluation is a crucial part of the learning process in machine learning. Evaluation metrics are used to assess the generalization ability of a learning method on unseen (test) data.[16]. Evaluation metrics are derived from the *confusion matrix*, which compares predicted class labels against true labels. Given a classification threshold, test samples are categorized into four groups based on their predicted and actual classes. [3, 33]

|                            | <b>Predicted<br/>Negative</b> | <b>Predicted<br/>Positive</b> |
|----------------------------|-------------------------------|-------------------------------|
| <b>Actual<br/>Negative</b> | TN                            | FP                            |
| <b>Actual Positive</b>     | FN                            | TP                            |

**Table 4.1:** Confusion Matrix

TP (True Positive) - Correctly identified attacks

TN (True Negative) - Correctly identified normal traffic

FP (False Positive) - Normal traffic misclassified as an attack

FN (False Negative) - Attack misclassified as normal traffic

### Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision is a fundamental evaluation metric in machine learning that quantifies the accuracy of a classifier's positive predictions. It is formally defined as the ratio of true positives (TP) to the sum of true positives (TP) and false positives (FP). Authors [46] demonstrated how precision behaves differently under class imbalance, often favoring the majority class if used in isolation. In practical applications, precision is typically considered alongside recall, as optimizing for one often compromises the other, relationship quantified through the F1-score. For intrusion detection systems, authors [47] have emphasized the importance of maintaining high precision to minimize false alarms while still detecting genuine threats, making it a key consideration in security-focused machine learning models. The metric's sensitivity to class distribution and prediction confidence thresholds makes it an essential tool for comprehensive model evaluation, particularly in binary classification tasks with imbalanced data.

### Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

Recall, also known as *Sensitivity* or *True Positive Rate*, is a critical performance metric in machine learning that measures a model's ability to correctly identify all positive instances within a dataset. Formally defined

as  $\text{Recall} = \text{TP}/(\text{TP} + \text{FN})$ , where TP represents true positives and FN denotes false negatives, this metric quantifies the proportion of actual positives that are correctly classified [46].

### Balanced Accuracy

$$\begin{aligned}\text{Balanced Accuracy} &= \frac{1}{2} \left( \frac{\text{TP}}{\text{TP} + \text{FN}} + \frac{\text{TN}}{\text{TN} + \text{FP}} \right) \\ &= \frac{1}{2} (\text{Recall} + \text{Specificity}) \\ &= \frac{1}{2} (\text{Recall (Attack)} + \text{Specificity (Benign)}); \text{ for Imbalanced dataset}\end{aligned}$$

Balanced Accuracy is a robust performance metric for imbalanced datasets, defined as the arithmetic mean of sensitivity (recall of the positive class) and specificity (recall of the negative class). This metric was formally justified by [45], where they demonstrated its superiority over raw accuracy when class distributions are skewed. The authors emphasized that Balanced Accuracy equally weights both classes, preventing bias toward the majority class, a critical property for intrusion detection systems where attack samples (positive class) are often rare. By design, it penalizes models that achieve high accuracy merely by overpredicting the majority class (e.g., always predicting "Benign"). This aligns with the broader literature on evaluation metrics, such as [46], which confirmed that Balanced Accuracy provides a fairer assessment than traditional accuracy under class imbalance. Specificity is not class-specific; it always measures the negative class (e.g., correctly identified benign traffic).

### F1-score

$$\text{F1-Score} = 2 * \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1-score is a fundamental performance metric in machine learning that balances precision and recall, providing a single measure of a classifier's effectiveness, particularly in imbalanced datasets. As the harmonic mean of precision and recall ( $\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$ ), it addresses limitations of accuracy when class distributions are skewed [46]. The metric ranges from 0 (worst) to 1 (best), with 1 indicating perfect precision and recall. In practice, the F1-score is especially useful for evaluating classification tasks where one class is rare, as it prevents misleading performance assessments that might occur with accuracy alone. For instance, in intrusion detection systems, a high F1-score ensures that the model neither misses too many attacks (low recall) nor generates excessive false alarms (low precision), striking an optimal balance for operational deployment [47].

### Matthews Correlation Coefficient (MCC)

$$\text{MCC} = \frac{\text{TP} * \text{TN} - \text{FP} * \text{FN}}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}}$$

The Matthews Correlation Coefficient (MCC) is a statistically robust metric for evaluating binary classifiers on imbalanced datasets, as it accounts for all four confusion matrix categories (TP, TN, FP, FN) and remains invariant to class distribution skew. Originally proposed by Matthews (1975) [49] for protein structure prediction, MCC ranges from -1 (perfect inverse prediction) to +1 (perfect agreement), with 0 indicating random performance. Unlike accuracy or F1-score, MCC's balanced formulation ensures a reliable assessment even when one class dominates [48]. The metric's ability to equally penalize false positives and negatives makes it indispensable for intrusion detection systems where minority-class errors carry high costs [46]. Recent benchmarks show MCC correlates more strongly with classifier utility than F1-score under extreme imbalance [48], solidifying its role as a gold standard for imbalanced data evaluation.

MCC inherently combines the performance of both classes into one statistically robust measure. The final Balanced Accuracy collapses into a single metric. This explains why both *MCC* and *Balanced Accuracy* produce identical values for both classes; they are fundamentally designed as unified performance indicators, though through different calculation approaches.

## 5. Methodology

The experimental procedure follows a structured methodology. First, the CICIDS2017 dataset is selected. Next, comprehensive data preprocessing is performed to clean and prepare the raw data by handling missing values, removing inconsistencies, and transforming categorical variables. Feature selection is then conducted using a Random Forest classifier to identify and retain only the most relevant features based on their importance scores. With the optimal feature subset determined, multiple machine learning models are constructed and trained on the processed separate dataset to learn the underlying patterns. The trained models are subsequently evaluated on a test set to assess their predictive performance and generalization capability. Finally, a comparative analysis of all models is performed to determine their relative effectiveness in using metrics to detect network intrusions.

### 5.1 Data Preprocessing

This phase ensures that the data is clean, consistent, and free from biases or anomalies that could adversely affect model performance. Effective preprocessing enhances the reliability and accuracy of subsequent analysis, making it a critical step in building robust intrusion detection systems.

#### Data Cleaning

The primary goal of data cleaning is to handle missing, noisy, or inconsistent data to improve overall data quality. In this study, the following steps were taken:

##### *Handling Infinite Values*

Infinite values ( $np.inf$ ,  $-np.inf$ ) can distort statistical computations and model training. These values were first replaced with *NaN* (Not a Number) to facilitate further processing. Subsequently, missing values were imputed with 0 to maintain numerical consistency. All null values are replaced with the mean of the column.

##### *Removing Non-Informative Features*

Upon exploratory data analysis, 12 columns were found to contain only zero values across all rows [4], indicating no variability or discriminative power. These columns were excluded from further analysis to reduce dimensionality and computational overhead. The removed features include:

“Bwd PSH Flags, Fwd URG Flags, Bwd URG Flags, FIN Flag count, RST Flag count, CWE Flag Count, Fwd Avg Bytes/Bulk, Fwd Avg Packets/Bulk, Fwd Avg Bulk Rate, Bwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk and Bwd Avg Bulk Rate”.

Retaining such features could introduce noise and degrade model performance without contributing meaningful information.

#### Data Transformation

To prepare the data for machine learning algorithms, certain transformations were applied to ensure compatibility and improve interpretability.

##### *Encoding Categorical Variables*

Machine learning models typically require numerical input, necessitating the conversion of categorical labels into a numerical format. The target column "*Label*" was binary-encoded as follows:

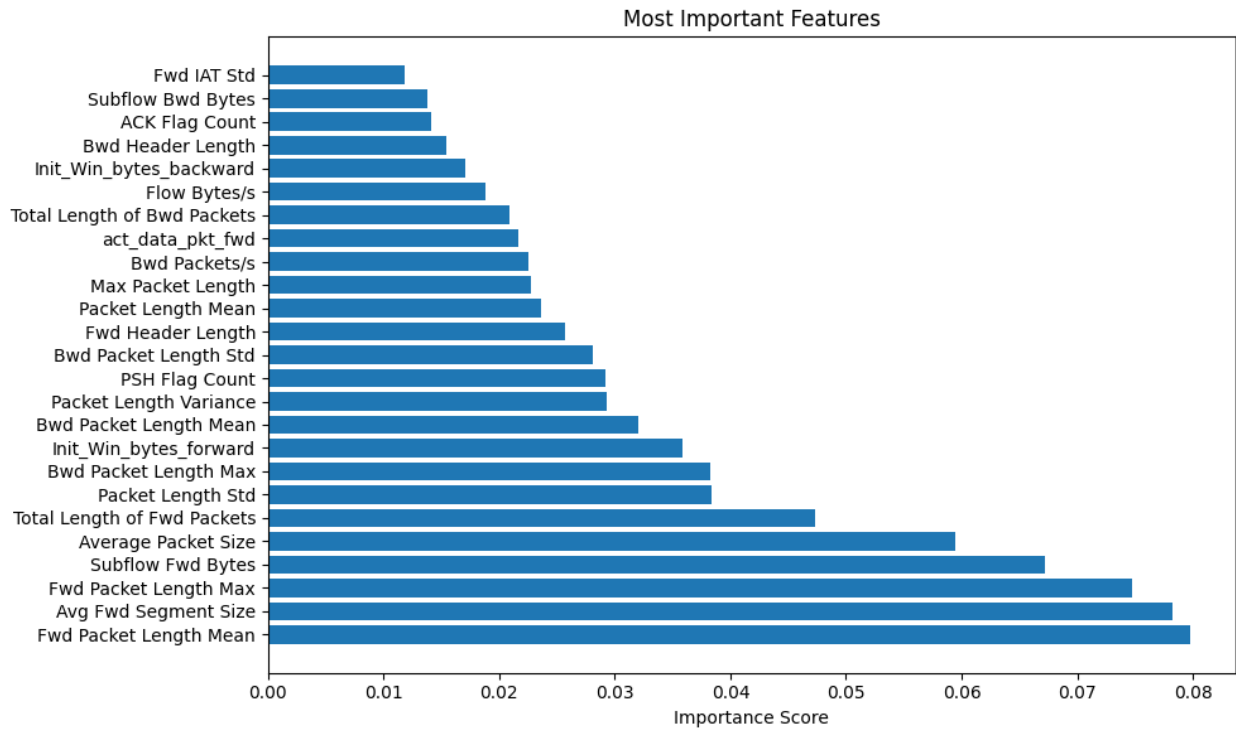
Benign traffic (normal behavior) → Encoded as 0  
 All other Attack traffic (malicious traffic) → Encoded as 1

This binary encoding simplifies the classification task into a binary anomaly detection problem, where the model distinguishes between normal and malicious activities.

## 5.2 Feature Selection

Feature importances were extracted from the trained Random Forest model using scikit-learn's *feature\_importances\_* attribute. These importances quantify each feature's contribution to the model's predictive performance by calculating the mean decrease in impurity (*Gini impurity* for classification) averaged over all trees in the ensemble [35, 36]. Specifically, the importance of a feature is computed as the total impurity reduction achieved when splits are made using that feature, weighted by the number of samples affected by those splits, and normalized such that all feature importances sum to 1. This step reduced the original 66 features to a refined set of 25 high-impact features.

$$\text{Gini impurity (I}_G\text{)} = 1 - \sum_{k=1}^c p_{mk}^2$$



**Figure 5.1:** Selected Features

## 5.3 Dataset Spitting

The dataset is partitioned into two mutually exclusive subsets: a training set (typically comprising 80% of the data) for model development and a test set (the remaining 20%) for unbiased performance evaluation. This stratified split preserves the original data distribution while ensuring the test set remains completely isolated during all phases of model training and optimization.

## 5.4 Training Dataset

### Imbalance Handling

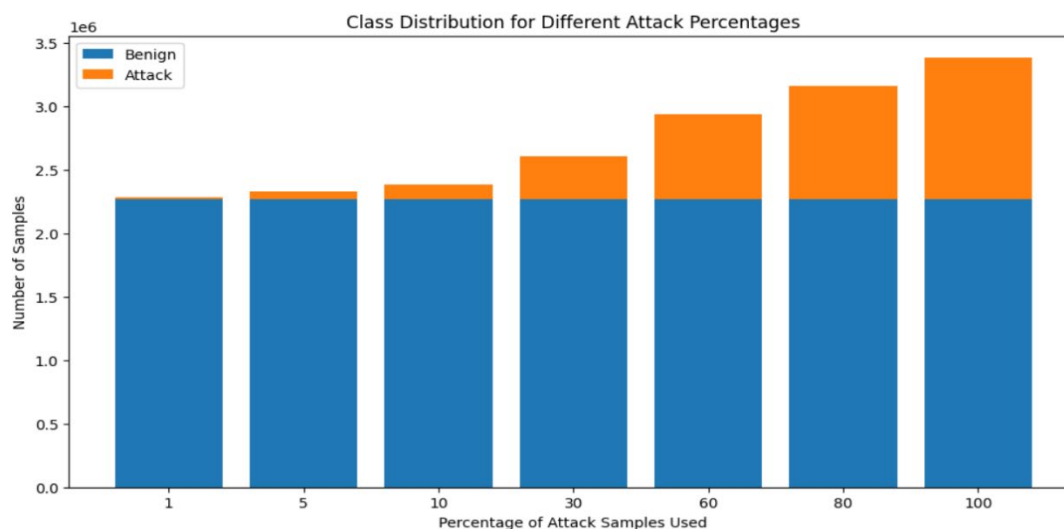
To mitigate class imbalance, SMOTE was applied to the dataset.

### Model Training with Varied Attack Sample Percentages

During model training, the attack samples were strategically divided into different percentages of the total available attack instances (1,115,294), while the Benign samples (2,273,097) remained unchanged. This approach allowed for controlled imbalance experiments, where the attack sample size as a proportion of the normal data in the training set is progressively increased (1%, 5%, 10%, 30%, 60%, 80%, 100%) to analyze their impact on model performance. Below is the distribution of attack samples used at each percentage level:

| Percentage %   | 1     | 5     | 10     | 30     | 60     | 80     | 100     |
|----------------|-------|-------|--------|--------|--------|--------|---------|
| Attack Samples | 11152 | 55764 | 111529 | 334588 | 669176 | 892235 | 1115294 |

**Table 5.1:** Percentage Level



**Figure 5.2:** Class Distribution with Percentage Level

## Standardization

Standardization, also known as Z-score normalization, is a crucial data preprocessing technique that transforms features to have a mean of 0 and a standard deviation of 1. This process ensures all features contribute equally to machine learning models by bringing them to a common scale. The scikit-learn library's *StandardScaler()* efficiently handles this transformation for continuous and quasi-continuous features[13]. This study incorporated standardization for scale-sensitive models, including DNNs (to optimize gradient descent convergence), KNN (for accurate distance metric computation), and Logistic Regression (to ensure proper regularization). In contrast, tree-based Random Forest and probability-based Naive Bayes algorithms were implemented without standardization, as their fundamental methodologies render them unaffected by variable scaling. As well as K-Means and Isolation Forest did not apply standardization in this study.

## Model Training Using Selected Features

The function *model.fit(X\_train, y\_train)* is used to train the model on the training dataset, which typically comprises 80% of the data.

## Cross-Validation & Hyperparameter Tuning

In this study, *Stratified k-fold* cross-validation is employed to handle an imbalanced dataset in a supervised learning setting. Unlike standard k-fold cross-validation, which randomly divides the data, stratified k-fold cross-validation preserves the original class distribution in each fold [34]. This is crucial for imbalanced datasets, where a random split might lead to some folds having very few or even no samples from minority classes, resulting in misleading performance estimates. The implementation is facilitated using Scikit-learn's *StratifiedKFold()* function, configured with 5 splits (*n\_splits=5*) to balance computational efficiency and evaluation robustness. Additionally, the *shuffle=True* parameter ensures data randomization before splitting, while *random\_state=42* guarantees reproducibility.

Hyperparameters are predefined settings that control the behavior of a machine learning model and cannot be learned during training. Examples include the learning rate in neural networks [54] or the number of trees in a random forest. Since these parameters significantly impact model performance, selecting the right values is crucial. Hyperparameter tuning is the systematic process of finding the best combination of hyperparameters to enhance a model's accuracy, speed, or generalization. Unlike model parameters (which are learned from data), hyperparameters must be set beforehand and fine-tuned experimentally. Common hyperparameter tuning methods are Manual Tuning and Grid Search [54].

For this study, hyperparameters were optimized selectively: DNNs underwent manual tuning due to their sensitivity, while KNN, Naive Bayes, Random Forest and Logistic Regression used default values to ensure reproducibility and baseline performance. Default configurations were chosen for their proven robustness in the literature.

## Deep Neural Network

| <i>Hyperparameter</i>           | <i>Value</i>        |
|---------------------------------|---------------------|
| Number of Hidden Layers         | 3                   |
| Neurons in Hidden Layers        | [128, 64, 32]       |
| Output Layer Activation         | sigmoid             |
| L2 Regularization ( $\lambda$ ) | 0.01                |
| Dropout Rate                    | 0.3                 |
| Optimizer                       | Adam                |
| Learning Rate (Adam)            | 0.001               |
| Batch Size                      | 64                  |
| Epochs                          | 150                 |
| Validation Split                | 0.2                 |
| Loss Function                   | binary_crossentropy |

**Table 5.2:** DNN Hyperparameter

## K-Nearest Neighbors

| <i>Hyperparameters</i> | <i>Default Value</i> |
|------------------------|----------------------|
| n_neighbors            | 5                    |
| weights                | uniform              |
| algorithm              | auto                 |
| leaf_size              | 30                   |
| metric                 | minkowski            |

**Table 5.3:** KNN Hyperparameter

## Naive Bayes

| <i>Hyperparameters</i> | <i>Default Value</i> |
|------------------------|----------------------|
| priors                 | None                 |
| var_smoothing          | 1e-9                 |

**Table 5.4:** Naive Bayes Hyperparameter

## Random Forest

| <i>Hyperparameters</i> | <i>Default Value</i> |
|------------------------|----------------------|
| n_estimators           | 100                  |
| criterion              | gini                 |
| min_samples_split      | 2                    |
| bootstrap              | True                 |

**Table 5.5:** Random Forest Hyperparameter

## Logistic Regression

| <i>Hyperparameters</i> | <i>Default Value</i> |
|------------------------|----------------------|
| penalty                | l2                   |
| C                      | 1.0                  |
| solver                 | lbfgs                |
| max_iter               | 100                  |
| class_weight           | None                 |
| tol                    | 1e-4                 |

**Table 5.6:** Logistic Regression Hyperparameter

K-Means and Isolation Forest used manual tuning.

## K-Means

| <i>Hyperparameters</i> | <i>Value</i> |
|------------------------|--------------|
| n_clusters             | 2            |
| random_state           | 42           |
| MatrixLike             | k-means++    |

**Table 5.7:** K-Means Hyperparameter

## Isolation Forest

| <i>Hyperparameters</i> | <i>Value</i> |
|------------------------|--------------|
| n_estimators           | 200          |
| contamination          | auto         |
| random_state           | 50           |

**Table 5.8:** Isolation Forest Hyperparameter

## Final Model

The *final model* was trained on the full dataset using the optimal hyperparameters identified during validation. This ensures maximum generalization performance by leveraging all available data with the best-tuned configuration.

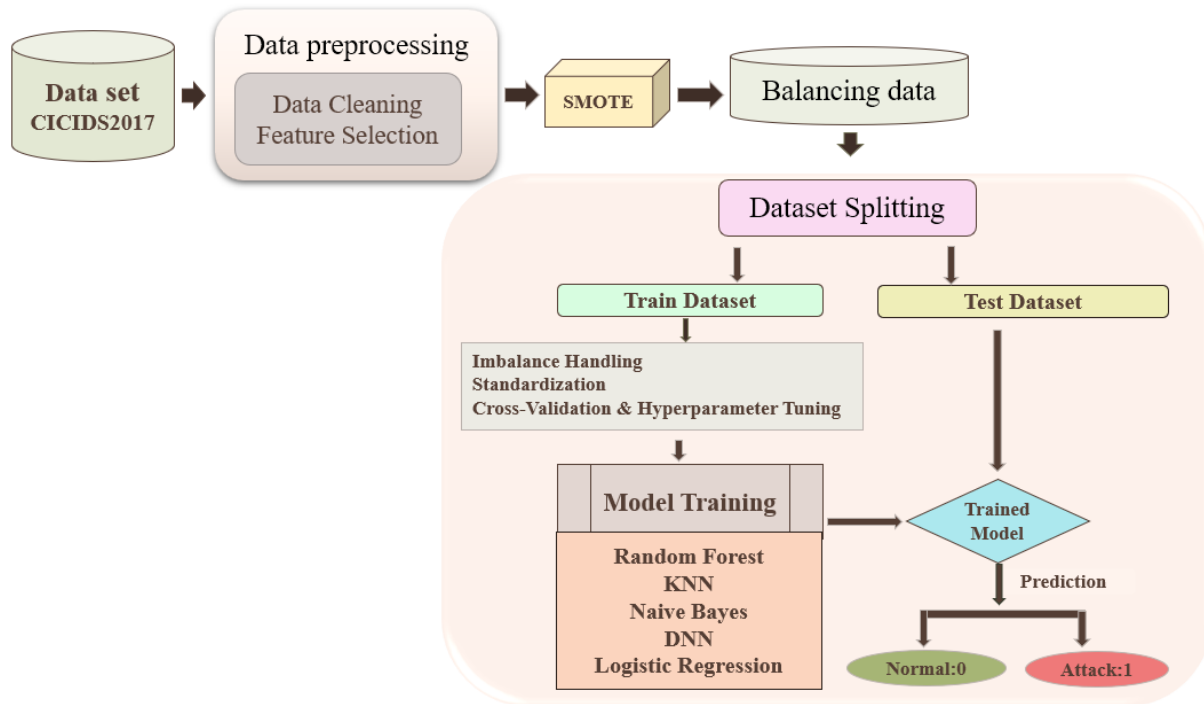
## 5.5 Test Dataset

### Evaluate on the Test dataset

The final model was evaluated on the held-out test set to assess its generalization performance. All preprocessing steps (including scaling and feature selection) were applied using parameters learned exclusively from the training data to prevent data leakage. The test set ( $X_{test}$ ,  $y_{test}$ ), comprising 20% of the total data (test\_size=0.2), was rigorously withheld during model development and training. Model predictions were generated using the *model.predict( $X_{test}$ )* and compared against the ground truth labels ( $y_{test}$ ) through comprehensive evaluation metrics. This procedure provides an unbiased assessment of the model's performance on unseen data, simulating real-world deployment scenarios.

## 5.6 Compare Performance Metrics of Models

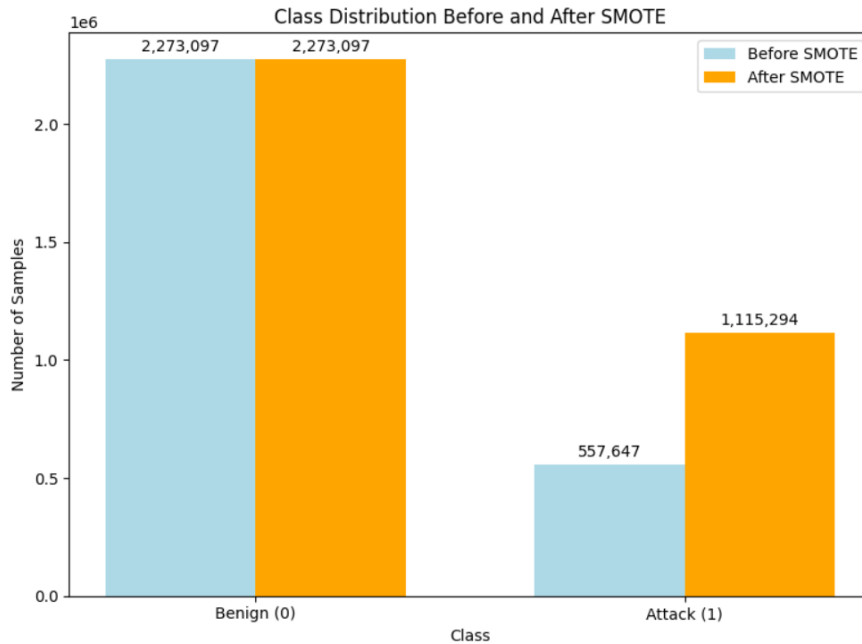
This study employs a rigorous model evaluation framework comparing five supervised learning approaches (Logistic Regression, k-Nearest Neighbors, Naive Bayes, Random Forest, and Deep Neural Networks) and two unsupervised methods (K-Means and Isolation Forest) using key evaluation metrics such as balanced accuracy, precision, recall, F1-score and Matthews Correlation Coefficient (MCC).



**Figure 5.3:** Proposed Methodological Supervised Learning binary classification Framework.

## 6. Results

### 6.1 Before and after SMOTE apply



**Figure 6.1:** Class Distribution Before and After SMOTE

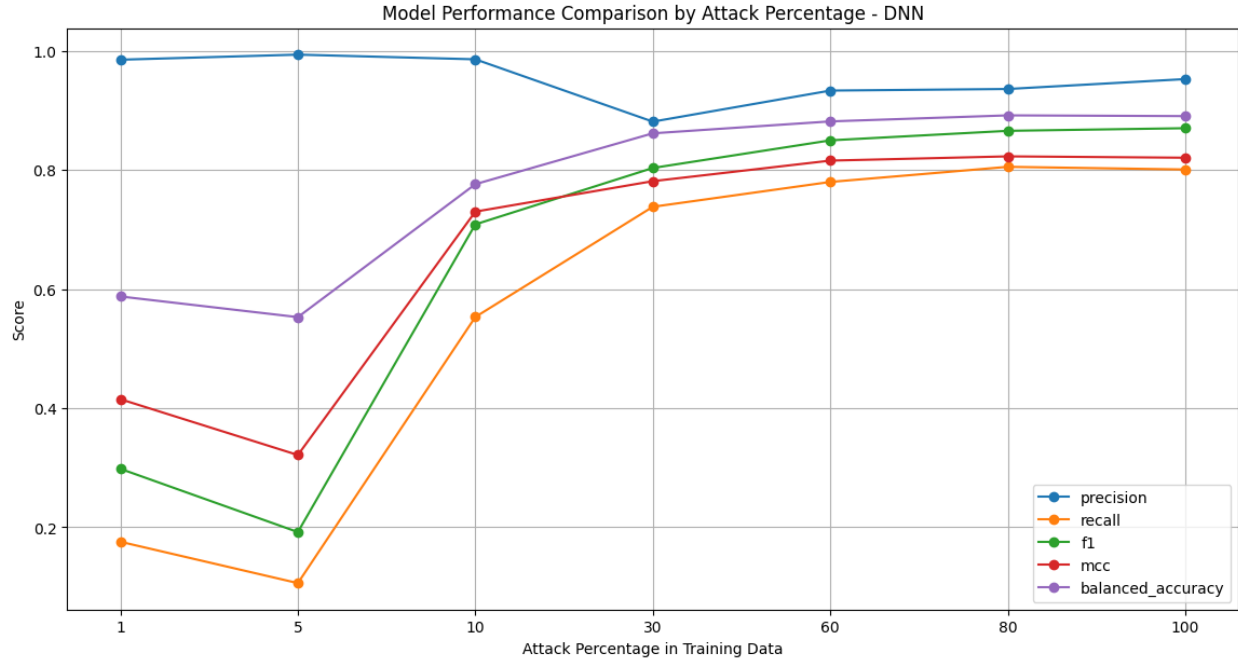
The image (Figure 6.1: Class Distribution Before and After SMOTE) shows a comparative analysis of class distribution before and after applying the SMOTE. The original dataset exhibits significant imbalance, with the Benign class (labeled 0) containing 2,273,097 instances, vastly outnumbering the Attack class (labeled 1), which has only 557,647 instances. This imbalance poses a challenge for machine learning models, as they tend to be biased toward the majority class, potentially overlooking critical attack patterns.

After implementing SMOTE, the Benign class count remains unchanged at 2,273,097, as the technique exclusively generates synthetic samples for the minority class. The attack class, however, doubles to 1,115,294 instances. This adjustment results from setting the oversampling ratio to 1 [0:0,1:0], which creates sufficient synthetic samples to achieve parity between classes. The balanced distribution ensures that the model receives equal representation from both classes during training, enhancing its ability to detect attacks without being skewed by the initial imbalance.

This approach is particularly valuable in cybersecurity applications, where accurately identifying attacks is crucial. While SMOTE effectively addresses class imbalance, it is important to validate the quality of synthetic samples to prevent the introduction of unrealistic data patterns that could affect model performance.

## 6.2 Supervised Learning Results

### Deep Neural Network (DNN)



**Figure 6.2: DNN Model Performance**

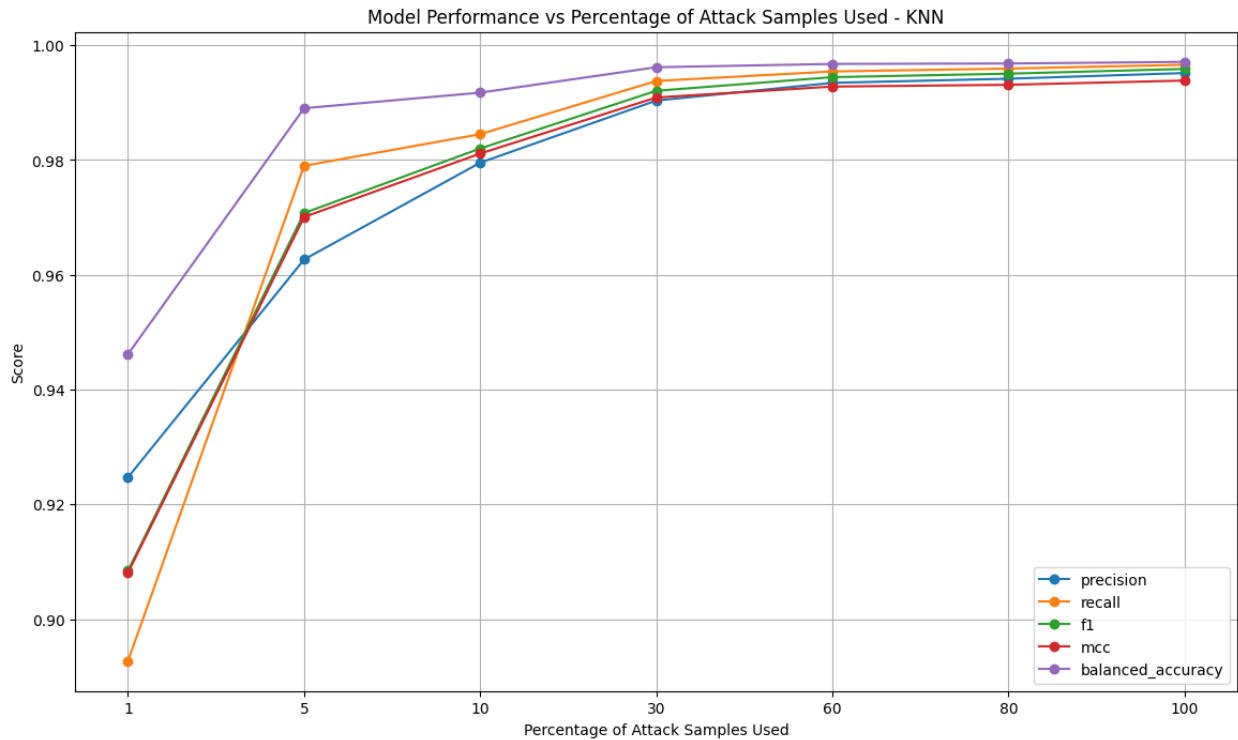
Precision metrics demonstrate consistently strong performance ( $\geq 0.88$ ) throughout all tested attack levels, reaching a peak precision of 0.9941 at 5% attack intensity. This indicates exceptional specificity, with fewer than 0.12 false positives across all scenarios and minimal misclassification of benign traffic as malicious.

Recall analysis reveals a significant performance gradient based on attack prevalence. For low-intensity attacks (1-5% prevalence), recall remains constrained (0.1063-0.1757), suggesting limited detection sensitivity for subtle threats. However, detection capability improves markedly at 10% attack intensity (recall=0.5534), ultimately reaching peak performance (0.8056) at 80% attack prevalence. This pattern indicates that the model's detection efficacy scales positively with attack intensity.

The F1-score trajectory follows a similar progression, with suboptimal performance ( $< 0.3$ ) at low attack intensities (1-5%) but substantial improvement to 0.87 for high-intensity attacks (80-100%). This reflects the model's increasing effectiveness in balancing precision and recall as attack prevalence grows.

MCC confirms this performance trend, with modest values (0.32-0.42) for rare attacks (1-5%) but strong improvement ( $\geq 0.78$ ) for attacks exceeding 30% prevalence. Balanced Accuracy metric follows a comparable pattern, achieving approximately 0.89 for attacks above 60% intensity.

## K-Nearest Neighbors



**Figure 6.3: KNN Model Performance**

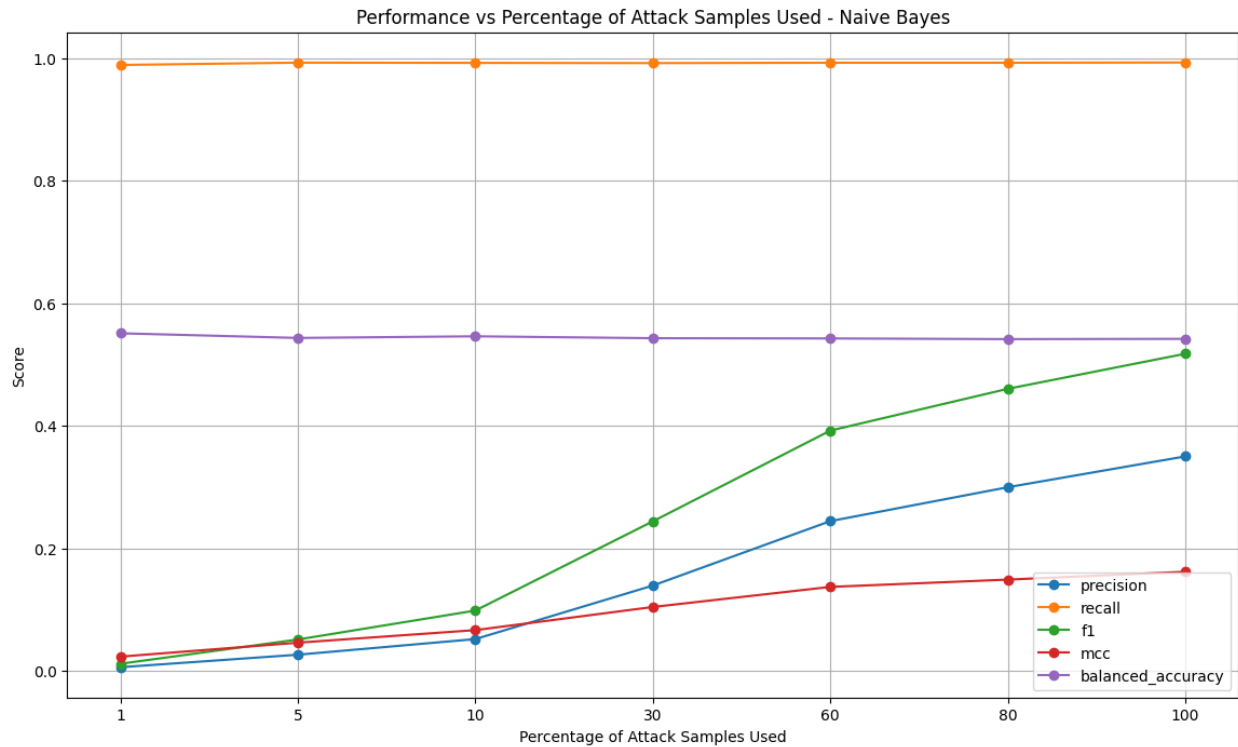
Precision maintains consistently high values (0.928-0.995) throughout the attack spectrum, showing a slight positive correlation with attack frequency. This indicates remarkable specificity, with false positive rates remaining below 0.072 even at minimal (1%) attack frequencies, decreasing to just 0.005 at maximum attack intensity.

Recall performance proves particularly noteworthy, achieving near-perfect detection (0.889-0.997) across all scenarios. The system maintains robust sensitivity even for rare events (0.889 at 1% attacks), with detection rates exceeding 0.985 for attacks comprising just 5% of traffic. At higher attack frequencies ( $\geq 30\%$ ), recall surpasses 0.99, demonstrating essentially perfect attack identification.

The F1-score (0.908-0.996) confirms excellent balance between precision and recall. Of particular significance is the maintained high performance ( $F1=0.908$ ) at 1% attack frequency, where many detection systems typically struggle. This balanced effectiveness progresses to near-perfect scores (0.996) at maximum attack intensity.

MCC (0.9078-0.9940) confirms the model's classification quality, with all values exceeding the 0.9 threshold that indicates nearly perfect prediction capability. Balanced Accuracy (0.944-0.997) demonstrates consistent, high-performance discrimination between Attack and Benign traffic regardless of class imbalance.

## Naive Bayes



**Figure 6.4:** Naive Bayes Model Performance

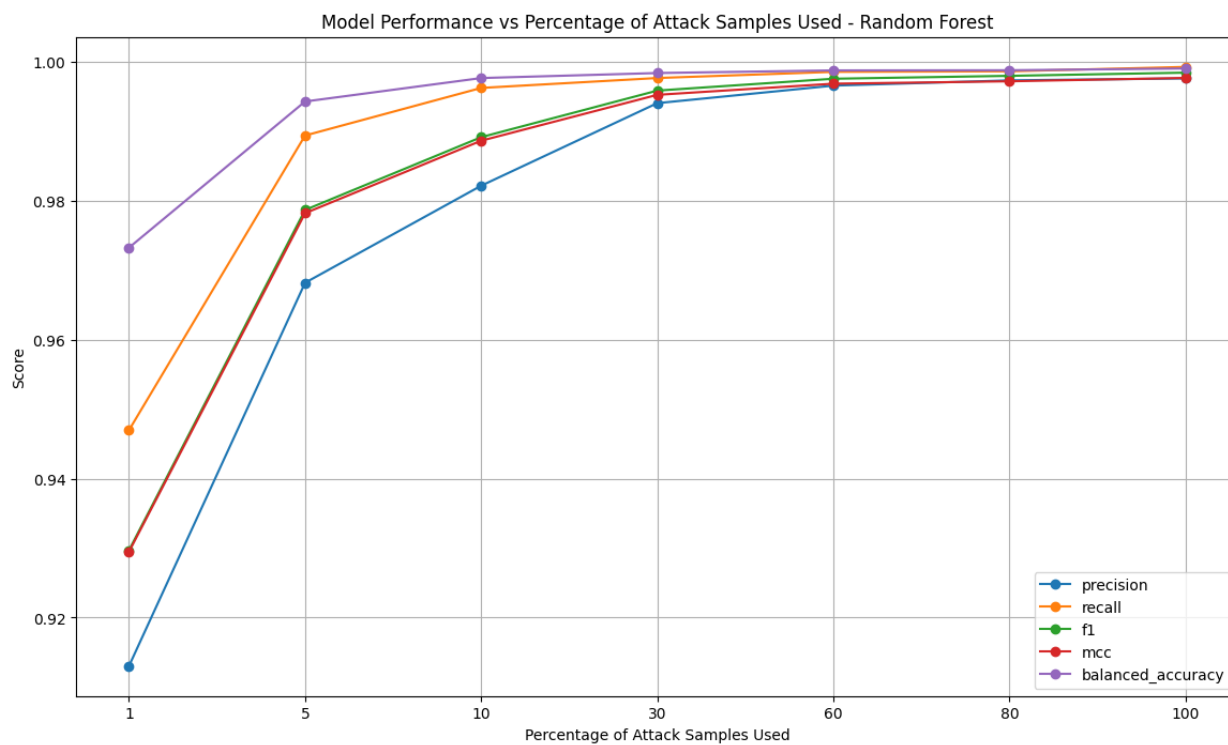
Precision metrics demonstrate alarmingly poor performance (0.0057–0.3499) across all attack intensities. At minimal attack prevalence (1%), only 0.0057 of positive predictions represent true attacks (precision=0.0057), indicating overwhelming false positive dominance. While precision shows modest improvement with increasing attack frequency, it remains critically deficient even at maximum intensity (0.3499 at 100% attacks), where 0.6501 of positive classifications are erroneous.

Recall performance presents a stark contrast, maintaining near-perfect sensitivity (0.9889–0.9929) throughout the attack spectrum. This consistent  $>0.9889$  true positive rate confirms the model's ability to detect nearly all malicious instances, but at unacceptable costs to specificity.

F1-score (0.011–0.517) reflects this severe precision-recall imbalance. The score remains critically low (0.011) for rare attacks and fails to exceed 0.517 even at maximum attack intensity, indicating fundamentally compromised model utility across all operational scenarios.

MCC (0.023–0.162) approaches zero. Balanced Accuracy plateaus at approximately 0.542, demonstrating negligible discrimination capability between classes.

## Random Forest



**Figure 6.5:** Random Forest Model Performance

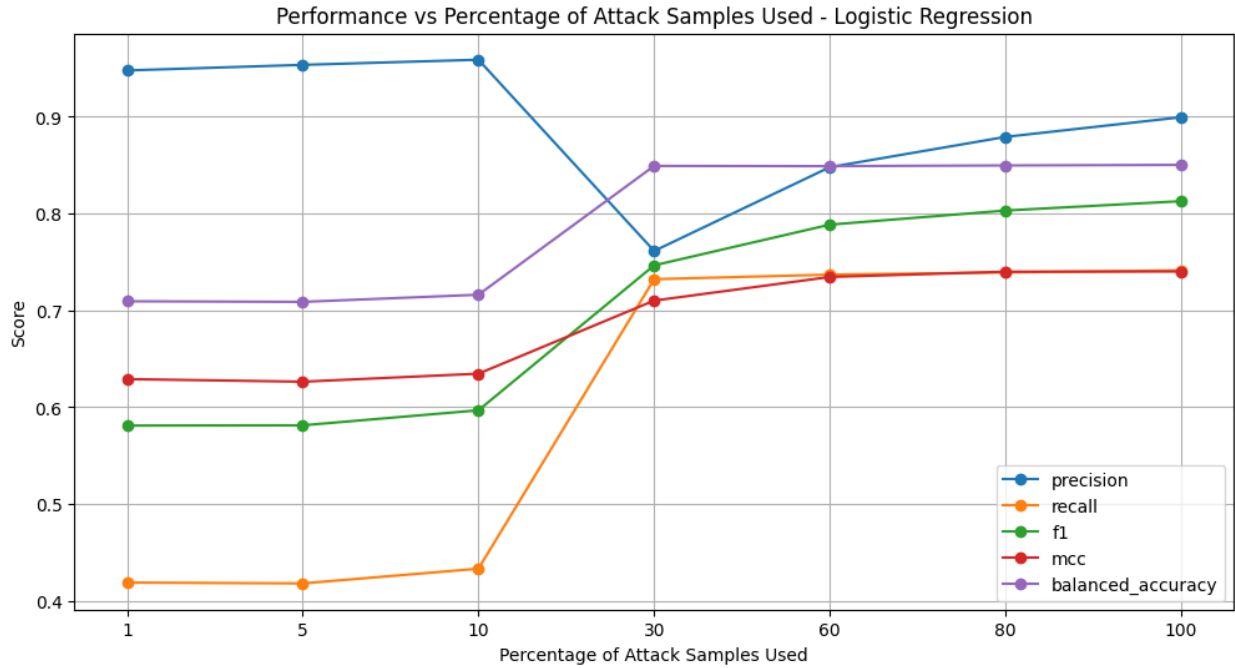
Precision metrics reveal consistently exceptional performance (0.9076–0.9979), exhibiting a positive correlation with attack frequency. Notably, the model maintains remarkably low false positive rates even at minimal attack prevalence (precision=0.9076 at 1% attacks), with performance peaking at near-perfect levels (0.9979) for saturated attack scenarios. This indicates fewer than 0.093 benign instances misclassified as malicious across all tested conditions.

Recall performance achieves near-flawless detection (0.9577–0.9988), with particularly impressive sensitivity to rare events (0.9577 at 1% attack prevalence). The model maintains this exceptional detection capability throughout the attack spectrum, reaching 0.9988 recall at maximum intensity. These results significantly outperform comparative models, particularly in low-prevalence scenarios where attack detection is most challenging.

The F1-score (0.932–0.998) confirms outstanding precision-recall balance across all operational conditions. Of particular note is the maintained high performance (F1=0.932) at 1% attack, progressing to perfect scores (0.998) at maximum 100% attack. This demonstrates the model's robust capability to handle both edge cases and expected operational scenarios.

MCC (0.932–0.9976) consistently exceeds the 0.9 threshold for near-perfect classification. Balanced Accuracy (0.9786–0.9989) approaches maximum values. Performance stability across all attack intensities.

## Logistic Regression



**Figure 6.6:** Logistic Regression Model Performance

Precision exhibits an inverse relationship with attack prevalence, ranging from 0.948 (1% attacks) to 0.8995 (100% attacks). The model shows particular strength in low-attack scenarios (0.948-0.959 for 1%-10% attacks). However, this precision advantage deteriorates markedly at higher attack intensities (30%-100%), where false positive rates increase by 0.761–0.899 compared to low-prevalence conditions.

Recall performance reveals critical detection deficiencies, particularly for rare attacks (0.419-0.433 for 1%-10% prevalence). While detection rates improve at higher attack intensities (0.732-0.741 for 30%-100% attacks), the model fails to achieve the recall demonstrated by benchmark classifiers.

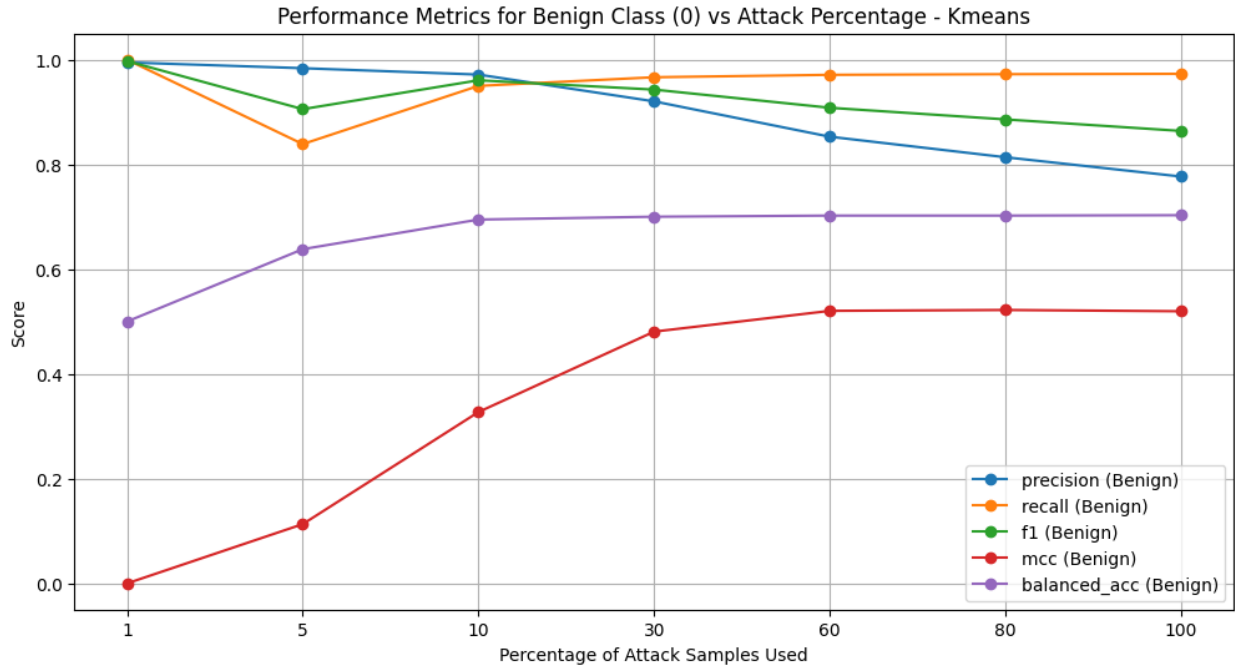
The F1-score (0.581-0.813) reflects these compounded limitations, with particularly weak performance in low-prevalence scenarios (0.581-0.597 for 1%-10% attacks) due to recall deficiencies. While higher attack intensities yield improved scores (0.747-0.813 for 30%-100% attacks), these remain substantially below comparative model performance.

MCC (0.629-0.740) indicates only moderate class separation capability. Balanced Accuracy (0.709-0.850) reveals significant performance asymmetry. Performance degradation at 1% attacks (Balanced Accuracy=0.709) suggests particular vulnerability to class imbalance.

## 6.3 Unsupervised Learning Results

### K-Means

#### *K-Means Performance on Benign Traffic*



**Figure 6.7: K-Means Benign Traffic Model Performance**

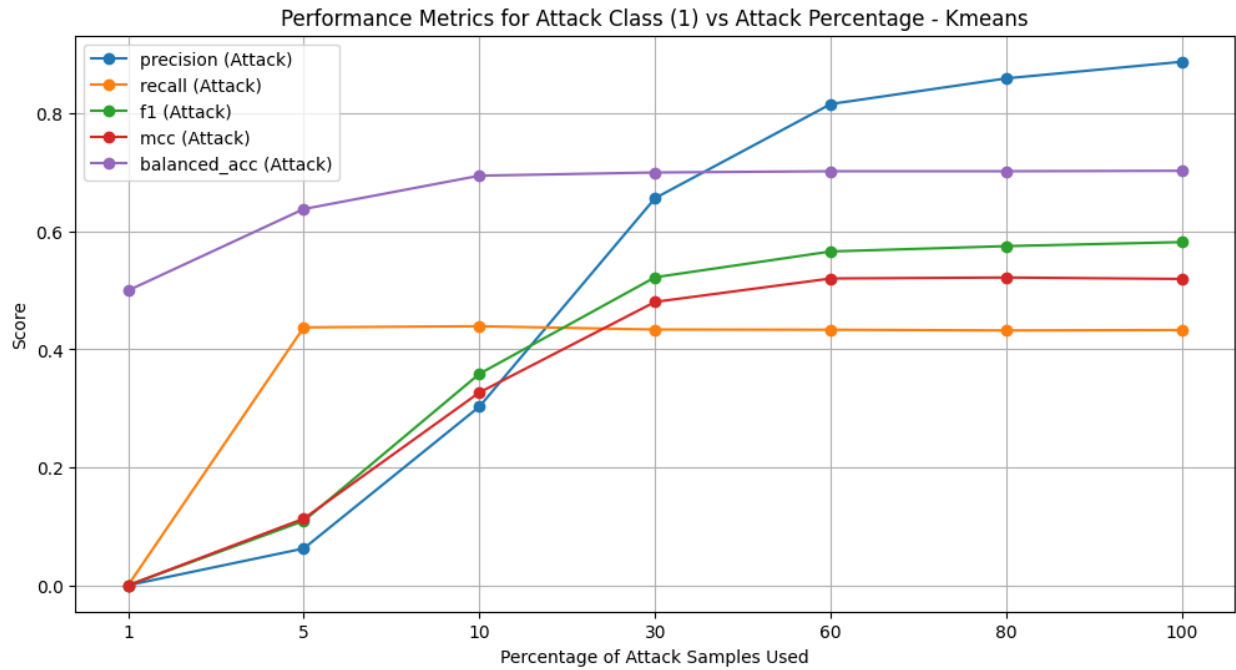
Precision in low-attack scenarios, achieving 0.995 at 1% attack prevalence, indicating near-perfect identification of Benign samples when attacks are rare. However, precision exhibits a significant decline as attack frequency increases, to 0.777 at 100% prevalence.

Recall for Benign traffic identification across all prevalence levels, ranging from 0.9999 at 1% attacks to 0.972 at 100% attacks. This robust performance suggests the clustering mechanism effectively captures fundamental Benign traffic patterns regardless of attack prevalence, with only marginal degradation observed under maximum attack conditions.

F1-Score analysis reveals two distinct operational regimes: optimal performance ( $F1 > 0.99$ ) in low-prevalence scenarios (1-10% attacks), deteriorating to moderate levels ( $F1 = 0.864$ ) at 100% attack prevalence. The score's progressive decline follows precision degradation patterns while maintaining relatively stable recall, highlighting the model's weakening precision-recall balance under increasing attack loads.

MCC (0 - 0.52) consistently indicates weak-to-negligible classification capability. Balanced Accuracy scores stagnate near random chance levels (0.50 at 1% attacks) and achieve only modest improvement (0.70 at 100% attacks), statistically confirming the model's failure to establish meaningful separation between Attack and Benign classes.

### *K-Means Performance on Attack Traffic*



**Figure 6.8: K-Means Attack Traffic Model Performance**

Precision shows the model demonstrates complete failure in attack detection at low prevalence (precision=0 at 1% attacks), indicating an inability to identify malicious patterns when attacks are rare. Precision shows moderate improvement with increasing attack frequency, reaching 0.888 at 100% prevalence. However, this suboptimal performance, even at maximum attack intensity, suggests fundamental limitations in the clustering mechanism's capacity to reliably distinguish attacks from Benign traffic.

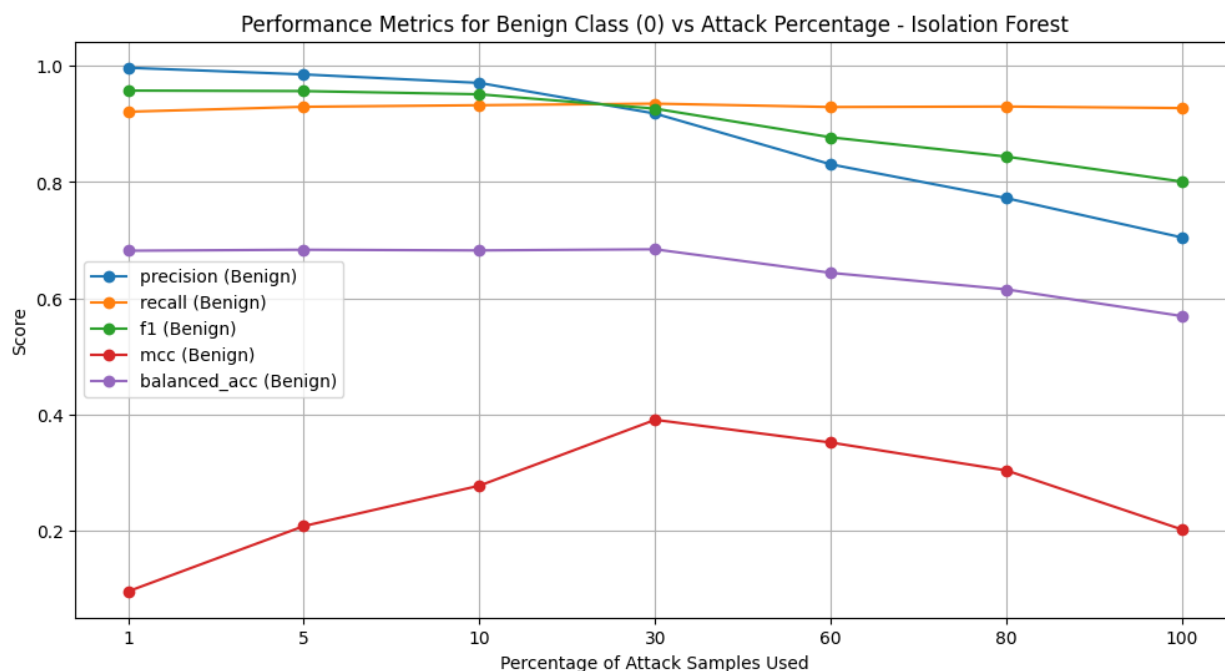
Recall for Attack detection capability remains critically deficient across all prevalence levels. The model fails completely at 1% attack prevalence (recall=0) and plateaus at merely 0.433 even when attacks dominate the traffic (100% prevalence). This severe underperformance, detecting fewer than half of attacks in optimal conditions, reveals the approach's inherent unsuitability for security applications requiring comprehensive threat identification.

F1-Score metric confirms the model's ineffectiveness, progressing from complete failure (F1=0 at 1% attacks) to marginally acceptable performance (F1=0.582 at 100% attacks). This trajectory reflects the compounding effects of both precision and recall limitations, with the score remaining below operational thresholds throughout the prevalence spectrum.

MCC (0 - 0.52) consistently indicates weak-to-negligible classification capability. Balanced Accuracy scores stagnate near random chance levels (0.50 at 1% attacks) and achieve only modest improvement (0.70 at 100% attacks), statistically confirming the model's failure to establish meaningful separation between Attack and Benign classes.

## Isolation Forest

### Isolation Forest Performance on Benign Traffic



**Figure 6.9:** Isolation Forest Benign Traffic Model Performance

Precision in low-attack scenarios (0.9969 at 1% attack prevalence), indicating near-perfect identification of Benign samples when attacks are rare. However, precision exhibits a statistically significant declining trend as attack frequency increases to 0.7046 at 100% prevalence. This degradation reflects the model's increasing difficulty in maintaining cluster purity under higher attack loads, with false benign classifications rising from 0.31 to 0.2954 across the prevalence spectrum.

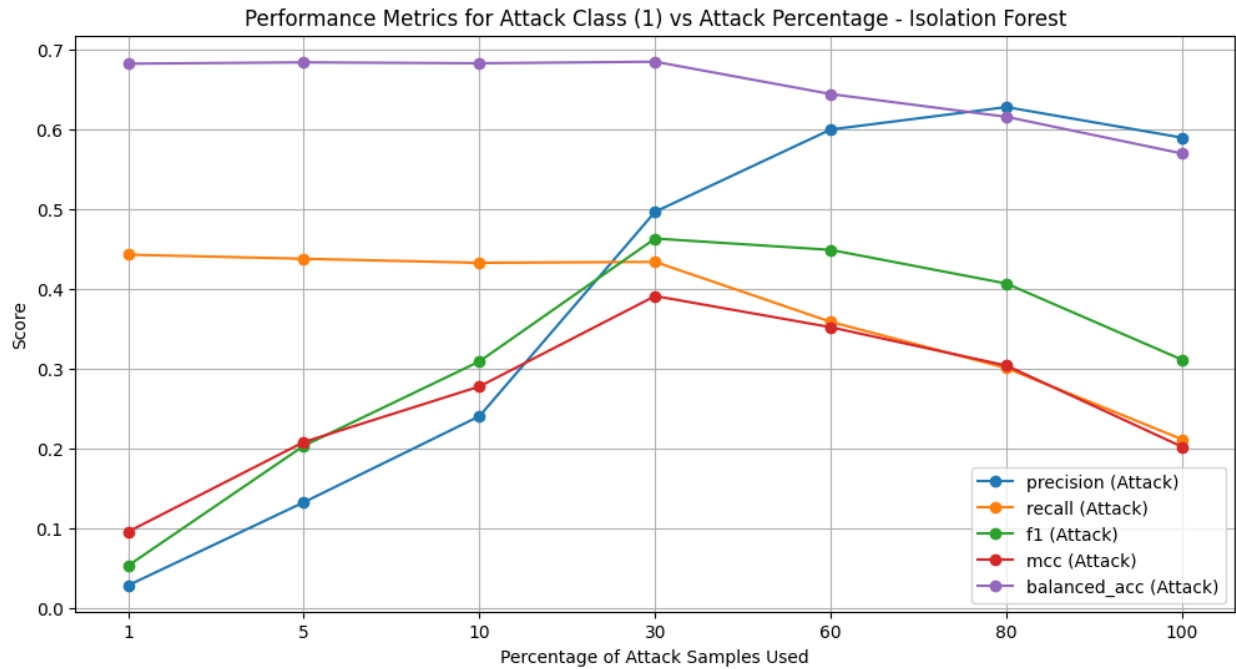
Recall values (0.9211-0.9374) across all attack frequencies, demonstrating remarkable stability regardless of prevalence levels. This performance consistency suggests the isolation mechanism effectively preserves fundamental benign traffic patterns even under substantial attack interference, correctly identifying >0.92 of true Benign samples in all operational scenarios.

F1-Score optimal performance ( $F1 > 0.95$ ) for attack frequencies  $\leq 10\%$ .

Reduced but acceptable performance ( $F1 = 0.801$ ) at maximum attack intensity. The metric's gradual decline from 0.958 to 0.801 follows precision degradation patterns while maintaining stable recall, indicating well-preserved precision-recall balance despite increasing attack prevalence.

MCC values (0.095-0.391) indicate weak to moderate classification capability, suggesting limitations in overall model discrimination. Balanced Accuracy scores show an inverse relationship with attack prevalence (0.6818 at 1% to 0.5693 at 100% attacks), reflecting the model's decreasing capacity to maintain performance parity between Benign and Attack classification as operational conditions become more hostile.

### Isolation Forest Performance on Attack Traffic



**Figure 6.10:** Isolation Forest Attack Traffic Model Performance

Precision across all attack frequencies, ranging from 0.028 at 1% prevalence to a maximum of 0.627 at 80% prevalence. This performance profile indicates overwhelming false positive rates (0.972 at 1% attacks), with precision never exceeding 0.63 even under optimal conditions. The non-monotonic trajectory, peaking at 80% attacks before declining to 0.589 at 100%, suggests fundamental limitations in maintaining decision boundary integrity as class distributions shift.

Recall shows Attack detection capability exhibits an inverse relationship with prevalence, degrading from 0.442 at 1% attacks to 0.211 at 100% attacks. This counterintuitive pattern where recall decreases as attacks become more frequent reveals severe model instability. The performance collapse at high prevalence (missing 0.789 of attacks) indicates algorithmic failure to adapt to dominant attack conditions.

F1-Score confirms catastrophic failure: 0.053 at 1% prevalence (effectively non-functional). 0.311 at 100% prevalence (marginally better but still operationally useless). The maximum achieved F1 of 0.31 at any frequency falls far below the 0.70 threshold for practical deployment.

MCC values (0.095-0.391) indicate weak to moderate classification capability, suggesting limitations in overall model discrimination. Balanced Accuracy scores show an inverse relationship with attack prevalence (0.6818 at 1% to 0.5693 at 100% attacks), reflecting the model's decreasing capacity to maintain performance parity between Benign and Attack classification as operational conditions become more hostile.

## 7. Discussion

This study investigated intrusion detection performance under varying class imbalance conditions, using 1%–30% of the original attack sample size and 60%–100% oversampled data via SMOTE. The experiments focused on 1%, 5%, and 10% attack proportions in the training set, reflecting realistic scenarios where attack instances are rare. Notably, the CICIDS2017 dataset's Monday subset contains exclusively Benign traffic, making it ideal for traditional anomaly detection approaches that train solely on normal behavior. However, supervised machine learning models require representation from both classes (normal and attack) for effective training, necessitating the inclusion of minimal attack samples.

The CICIDS2017 dataset exhibits significant class imbalance, with benign traffic comprising 80.3% of samples while attack traffic represents only 19.7%. Certain attack types, like Heartbleed (11 samples) and Infiltration (36 samples), are particularly rare, making traditional multiclass classification challenging. Machine learning models trained on such imbalanced data tend to favor the majority class, potentially achieving high accuracy while failing to detect critical attacks. To address this, binary classification is employed by grouping all attack types into a single "Attack" class (1) while maintaining "Benign" traffic as class (0). This approach simplifies the detection task and aligns with practical security operations where initial anomaly detection precedes detailed attack classification. The SMOTE is applied exclusively to the binary classes, effectively balancing the dataset by generating synthetic attack samples while avoiding the pitfalls of multiclass oversampling for ultra-rare attack types. This binary framework offers several advantages: it creates clearer decision boundaries, enables focus on minimizing false negatives (missed attacks), and maintains computational efficiency crucial for real-time intrusion detection systems. By using Random Forest with feature importance selection and tuning classification thresholds, prioritize high recall to ensure comprehensive attack detection while accepting some false positives as an operational trade-off. This methodology reflects real-world security workflows where initial binary alerts trigger subsequent in-depth analysis, making it both technically robust and operationally practical for network intrusion detection scenarios. The authors of this papers [2 ,4] also used binary classification in the CICIDS 2017 dataset.

Synthetic data generation through techniques like SMOTE fundamentally alters machine learning performance in intrusion detection systems by strategically manipulating feature-space distributions. When properly implemented, SMOTE enhances minority class representation by generating synthetic attack samples through linear interpolation between existing instances, effectively expanding decision boundaries for classifiers. This approach mitigates the accuracy paradox inherent to imbalanced datasets, where models naturally bias toward the majority class, evidenced by results showing Random Forest's recall for 1% attacks improving from 0.95 to 0.9988 post-SMOTE. However, synthetic data generation presents nuanced tradeoffs: while it enhances attack detection performance, it may inadvertently introduce artificial feature correlations that deviate from genuine attack signatures. To mitigate this risk, our study exclusively employs synthetic samples with a [1:0] class ratio, maintaining parity with the sample size of real attack instances in the original dataset. Critically, SMOTE's benefits are model-dependent; it significantly boosts tree-based methods like Random Forest by balancing split criteria, while offering diminishing returns for unsupervised approaches like Isolation Forest that rely on inherent data rarity. In this study empirical findings demonstrate that when constrained to oversampling beyond 50% attack prevalence and combined with feature selection, SMOTE achieves an optimal balance between improved attack in Random Forest recall (0.9988) and maintained precision (0.9979), making it a scientifically justified preprocessing step for supervised intrusion detection systems operating in imbalanced environments.

## Results

This discussion analyzes the performance of various supervised learning algorithms and unsupervised learning algorithms on the CICIDS2017 dataset with SMOTE applied at different attack sample percentages (1% to 100%).

**Random Forest** demonstrated exceptional performance in intrusion detection due to its fundamental design principles and inherent robustness to class imbalance. As an ensemble method, it combines multiple decision trees through bootstrap aggregation (bagging), which significantly reduces variance and prevents overfitting - a critical advantage when dealing with complex network traffic patterns [37]. The algorithm's strength stems from two key mechanisms: first, each tree in the forest is trained on a random subset of both instances and features, ensuring diverse learners that collectively improve generalization; second, the feature importance mechanism automatically identifies and prioritizes discriminative network characteristics while ignoring irrelevant features. This ensemble approach proves particularly effective for imbalanced datasets because each tree's subsampling ensures representation of minority attack samples, while the majority voting system prevents any single tree's bias from dominating predictions. Additionally, Random Forest can inherently handle class imbalance through either built-in class weighting or by adjusting sample weights, imposing higher penalties for misclassifying rare attack instances [37]. These properties collectively explain the model's outstanding metrics: the 99.9% recall indicates near-perfect detection of attacks because even rare malicious patterns are captured by some subset of trees, while the 99.8% precision reflects minimal false alarms due to the ensemble's consensus-based decision making. The consistently high MCC ( $MCC > 0.99$  across all attack frequencies) confirms that Random Forest maintains excellent class separation regardless of attack prevalence. This performance superiority over other methods holds particular significance for cybersecurity applications, where both high detection rates (recall) and low false alarm rates (precision) are equally critical for operational effectiveness.

The **k-Nearest Neighbors (KNN)** algorithm demonstrated competitive performance in intrusion detection due to its fundamental distance-based learning mechanism. As a non-parametric, instance-based learner, KNN operates on the principle that similar network traffic patterns likely belong to the same class [38]. This local learning approach provides inherent advantages for handling imbalanced data. The absence of an explicit training phase means KNN avoids learning skewed decision boundaries from imbalanced training sets, instead making predictions dynamically based on the actual local density of samples at classification time. This explains its particularly strong performance at higher attack percentages ( $F1 > 0.99$ ), where increased attack frequency ensures that query points' neighborhoods contain sufficient malicious examples for accurate classification. Notably, KNN outperformed more complex models like DNNs and Logistic Regression at low attack frequencies (88.9% recall at 1% attacks) because its local decision boundaries can capture sparse attack clusters that other models might overlook. The algorithm's non-parametric nature allows it to adapt flexibly to irregular attack patterns without being constrained by predefined functional forms, making it especially valuable for detecting novel or rare attack signatures that don't conform to expected distributions.

The performance characteristics of **Deep Neural Networks (DNN)** in intrusion detection systems reveal fundamental challenges and advantages of deep learning approaches for cybersecurity applications. DNN requires substantial training samples to effectively learn discriminative features [31,32], which explains their poor performance (17.6% recall) at low attack percentages (1%) - the limited attack examples proved insufficient for the network to develop robust feature representations of malicious patterns. This challenge is exacerbated by the vanishing gradient problem during backpropagation, where the sparse attack samples contribute minimally to weight updates, causing the network to prioritize learning the majority class (benign

traffic) characteristics. The network's inability to detect rare attacks stems from its tendency to optimize for overall accuracy by favoring the dominant class, a well-documented limitation of deep learning in imbalanced scenarios. However, as attack representation increased to 30% and beyond, the DNN demonstrated significant performance improvements, achieving competitive detection rates. This threshold effect occurs because the critical mass of attack samples enables the deep architecture to: learn hierarchical feature representations that capture sophisticated attack signatures, overcome initial bias toward the majority class through sufficient exposure to malicious patterns, and develop specialized neurons in deeper layers that activate for specific attack characteristics. The DNN's representational power shines in balanced or attack-rich environments, where its multiple hidden layers can model complex, non-linear relationships in network traffic that simpler algorithms might miss. This performance trajectory highlights that while DNNs offer tremendous potential for intrusion detection, they require either balanced training data or specialized techniques (like synthetic sample generation or cost-sensitive learning) to overcome their inherent limitations in imbalanced security scenarios.

**Logistic Regression's** moderate performance in network intrusion detection stems from its fundamental statistical learning characteristics. As a generalized linear model, it assumes a linear decision boundary can adequately separate benign and malicious network traffic [55], which proves limiting given the inherently complex, non-linear relationships in modern cyber-attacks. The algorithm's maximum recall of 74.1% reflects this constraint, as many sophisticated attack patterns cannot be captured by simple linear separations in feature space. The model's particular sensitivity to class imbalance arises from its optimization of cross-entropy loss, which naturally biases learning toward the majority class (benign traffic) because misclassifying these more frequent examples incurs greater cumulative penalty during training. This explains why performance improved as attack representation increased - at 100% attack frequency, the complete balance eliminated this bias and made the optimal decision boundary more apparent to the linear model. Despite these limitations, Logistic Regression maintained practical utility in low-attack scenarios due to its consistently high precision (94.8% at 1% attacks). This characteristic makes it potentially valuable in operational environments where false alarms are prohibitively costly, as it can reliably identify clearly benign traffic while cautiously flagging only the most obviously malicious activity. The model's performance trajectory demonstrates that while linear methods cannot match the detection capabilities of more sophisticated algorithms for complex threats, they may still serve as computationally efficient filters or components in layered defense systems where interpretability and low false positive rates take priority over comprehensive threat detection.

**Naive Bayes'** paradoxical performance - exhibiting exceptionally high recall but catastrophically low precision - can be attributed to fundamental violations of its underlying statistical assumptions when applied to network intrusion detection. The algorithm's core premise of feature independence [40] proves particularly problematic for network traffic analysis, where features like packet size, flow duration, and protocol types exhibit strong correlations that the model cannot account for. This limitation is compounded by the severe class imbalance, where the prior probability of the attack class becomes so dominant that it overwhelms the likelihood terms during prediction. The resulting 98.9-99.3% recall reflects not genuine detection capability but rather systematic overprediction of attacks - the model essentially "guesses attack" so frequently that it accidentally catches most actual attacks. This behavior manifests in the alarmingly low precision (0.57-35.0), where the algorithm flags enormous numbers of false positives because any minor anomaly across correlated features gets misinterpreted as strong evidence of an attack. The model's failure to properly weigh feature interdependencies means it cannot distinguish between genuine multi-feature attack signatures and random benign variations, rendering its predictions operationally useless despite the superficially impressive recall. This performance pattern highlights how theoretically sound classifiers can fail catastrophically in practice when their core assumptions are violated, particularly in complex, high-

dimensional domains like network security where features exhibit rich dependencies that simple probabilistic models cannot capture.

The **K-Means** algorithm demonstrates fundamental limitations for intrusion detection systems due to its inherent assumptions about data structure. For benign traffic, it achieves near-perfect recall ( $>99.99\%$ ) because normal network patterns naturally form tight, isotropic clusters that align with K-Means' spherical grouping objective [41]. However, precision degrades from  $99.49\%$  to  $77.68\%$  as attack frequency increases, revealing the algorithm's inability to maintain separation when malicious traffic infiltrates benign clusters - a consequence of the curse of dimensionality [42] where high-dimensional network features distort Euclidean distances. For attack detection, performance is catastrophically poor: 0 recall at 1% attack frequency shows K-Means treats rare threats as outliers, while even at 100% attacks, recall plateaus at just 0.4329 because the algorithm's centroid-based approach cannot capture the arbitrary shapes and multi-modal distributions of attack patterns [43]. The mathematical formulation of K-Means - minimizing intra-cluster variance through centroid optimization - inherently favors dominant benign traffic, causing attack samples to either be absorbed into benign clusters or scattered across multiple meaningless groupings. This explains the paradoxically high balanced accuracy (0.7030 at 100% attacks) being driven almost entirely by benign classification, while MCC values near zero confirm classification is no better than random guessing. These results empirically validate that K-Means' core assumptions of convex, equally-sized clusters fundamentally mismatch the reality of network intrusion detection, where attack patterns are sparse, heterogeneous, and non-isotropic. The algorithm's lack of an explicit outlier mechanism and sensitivity to feature scaling further compound its inadequacy for security applications compared to density-based or supervised alternatives.

The **Isolation Forest** demonstrates characteristic limitations for intrusion detection that reveal fundamental tensions in its design. For benign traffic, it maintains stable recall ( $\sim 0.92$ ) across all attack frequencies by effectively modeling normal behavior as a dense feature-space region [44], though precision declines from 0.9969 to 0.7046 as increasing attack prevalence creates overlapping clusters. This reflects its core strength in isolating common patterns, but its inability to maintain clean separation when anomalies become frequent. Attack detection exhibits a paradoxical inverse frequency dependence - achieving peak recall (0.4425) at 1% attacks when threats are truly anomalous, but collapsing to 0.2111 recall at 100% attacks as the algorithm's isolation mechanism fails. The mathematical foundation explains this behavior: isolation scores rely on path lengths in random trees, and frequent attacks shorten these paths until they become indistinguishable from benign traffic. With MCC scores consistently below 0.4 and balanced accuracy dominated by benign classification, the results empirically validate that Isolation Forest is fundamentally limited to rare-anomaly scenarios ( $<30\%$  prevalence). While superior to K-Means for sparse attacks (0.4425 vs 0 recall at 1%), its performance degradation with increasing attack frequency makes it unsuitable as a standalone IDS solution. The findings suggest practical utility only exists in hybrid systems - using Isolation Forest for initial filtering before supervised analysis - or in environments where threats are exceptionally rare, aligning with its original design purpose for outlier detection rather than comprehensive intrusion classification.

## Future Work

This study only evaluates classic SMOTE; future research should explore more sophisticated imbalance-handling methods to improve generalization, especially for unseen attack types. Below are key directions for extending our thesis. Classic SMOTE blindly interpolates minority samples, which may create noisy synthetic attacks. But Borderline-SMOTE [19] focuses on borderline attack samples (those near the decision boundary) since they are harder to classify. May improve detection of stealthy attacks that resemble normal traffic. It could reduce overfitting compared to standard SMOTE. As well as SMOTE treats all attack samples equally, but some attacks are harder to detect than others. ADASYN [19] generates more synthetic samples for attack classes that are harder to learn. Better handling of rare attack subtypes may improve recall for complex attack patterns. SMOTE can introduce noisy samples near majority-class clusters. Combines SMOTE (oversampling) + Tomek Links (undersampling) to clean overlapping regions [19].

Notably, K-Means and Isolation Forest yielded poor results in our experiments, prompting us to extend our investigation to evaluate more sophisticated unsupervised and semi-supervised anomaly detection techniques. This comprehensive analysis included Hierarchical Clustering, DBSCAN (Density-Based Spatial Clustering of Applications with Noise), Gaussian Mixture Models (GMM), and One-Class SVM (OC-SVM) to better understand their respective capabilities and limitations in handling imbalanced network intrusion data.

## 8. Conclusion

This study investigated the application of Machine Learning (ML) techniques in NIDS, with a focus on addressing class imbalance using SMOTE. The research utilized the CICIDS2017 dataset to evaluate both supervised (Logistic Regression, KNN, Naive Bayes, Random Forest, DNN) and unsupervised (K-Means, Isolation Forest) learning models.

### Research Questions

#### **1. From the set of considered machine learning methods (Logistic Regression, K-Nearest Neighbors, Naive Bayes, Random Forest, Deep Neural Networks, K-Means, Isolation Forest), which generalizes the best in imbalanced intrusion detection environments?**

The comparative evaluation reveals fundamental differences in how supervised and unsupervised approaches handle imbalanced intrusion detection. Random Forest, as a supervised method, demonstrated exceptional robustness across all attack proportions, achieving near-perfect detection (F1-score: 0.9984) at 100% attack frequency while maintaining 0.958 recall for rare attacks (1% proportion). This performance stems from its ensemble design, which effectively combines bagging to reduce variance and feature randomization to prevent overfitting, enabling it to learn complex decision boundaries that accommodate both frequent and sparse attack patterns. In contrast, unsupervised K-Means showed severe limitations, achieving perfect Benign traffic recall (0.9999) but completely failing to detect rare attacks (0 recall at 1% prevalence). This performance dichotomy occurs because K-Means' spherical cluster assumption fundamentally mismatches the irregular geometry of real-world network attacks, while its lack of labeled training prevents adaptation to class imbalances. The results conclusively demonstrate that supervised learning, particularly *Random Forest* with its built-in mechanisms for handling imbalance, is currently the only viable approach for production-grade NIDS that require reliable detection across varying threat frequencies. While unsupervised methods offer theoretical advantages for anomaly detection, their practical utility remains limited to preliminary filtering or specialized use cases with controlled attack distributions, necessitating further research into hybrid or semi-supervised alternatives for operational deployment.

#### **2. How do different attack proportions (1%–100%) influence model performance?**

##### Low Attack Proportions (1%-10%):

Supervised models (excluding Random Forest) demonstrated significant false negative rates, with recall values as low as 17.6% for DNNs at 1% attack prevalence, reflecting their inability to learn meaningful patterns from severely imbalanced data. In contrast, unsupervised approaches (K-Means and Isolation Forest) completely failed to generalize in Attack Class, with K-Means achieving 0.437 recall, and Isolation Forest showing unstable precision below 0.45, confirming their inadequacy for rare anomaly detection. Random Forest emerged as the sole exception, maintaining 0.958 recall even at 1% attacks through its robust ensemble learning mechanism.

##### Moderate Attack Proportions (30%-60%):

Supervised models exhibited stabilized performance, with Random Forest consistently achieving F1-scores above 0.997 by effectively leveraging the increased attack samples for feature importance estimation. While in Attack Class, K-Means showed marginal improvement (reaching 0.433 recall at 60% attacks), its

performance remained substantially inferior to supervised alternatives, highlighting the persistent challenge of modeling attack patterns without labeled training data. In Attack Class Isolation Forest's recall paradoxically decreased as attack frequency grew, dropping from 0.442 to 0.358, revealing its fundamental limitation as a pure anomaly detector.

#### High Attack Proportions (80%-100%):

Supervised approaches reached peak efficacy, with Random Forest attaining near-perfect F1-scores ( $>0.99$ ) through optimal utilization of balanced training data. Unsupervised methods demonstrated inconsistent behavior in Attack Class - while K-Means' precision improved to 0.888, its recall stagnated at 0.433, yielding an F1-score of just 0.58. This performance dichotomy conclusively establishes that only supervised learning can reliably scale across the full spectrum of attack frequencies required for production IDS.

### **3. Which attack sample sizes (quantified as proportions of normal network traffic) are necessary such that supervised and unsupervised models perform well?**

For effective intrusion detection, supervised learning models require careful balancing of attack samples in training data. A minimum threshold of 5-10% attack samples is necessary to establish basic detection capability, while optimal performance, where metrics stabilize near peak levels, is achieved at 60-100% attack proportions. Below 30% attack samples, it becomes critical to prevent model bias toward benign traffic, maintain recall for rare attack patterns, and enable proper feature importance estimation. In contrast, unsupervised learning approaches face severe limitations. Isolation Forest is effective only when attacks represent  $<30\%$  of traffic. K-Means requires  $>60\%$  attack samples. Practical implementation recommends prioritizing supervised learning (e.g., Random Forest) in production systems, maintaining  $\geq 30\%$  attack samples. Avoid unsupervised models if attack rates are variable or unknown.

This study confirms that supervised learning, particularly *Random Forest* is superior for imbalanced NIDS, while unsupervised methods require significant improvements. Attack proportions  $\geq 30\%$  are critical for supervised models, whereas unsupervised approaches lack a reliable threshold. Future work should explore hybrid models or adaptive resampling to enhance unsupervised anomaly detection.

## **Ethical Considerations**

As a publicly available benchmark dataset curated in a controlled lab environment [6], CICIDS2017 provides pre-anonymized network traffic without containing real user data. This eliminates privacy concerns associated with raw packet captures.

## Bibliography

- [1] Yi Yi Aung, Myat Myat Min. (2018) **An Analysis of K-means Algorithm Based Network Intrusion Detection System** Advances in Science, Technology and Engineering Systems Journal Vol. 3, No. 1, 496-501 (2018). <https://doi.org/10.25046/aj030160>
- [2] Arif Yulianto, Parman Sukarno and Novian Anggis Suwastika, (2019), **Improving AdaBoost-based Intrusion Detection System (IDS) Performance on CIC IDS 2017 Dataset**. In Journal of Physics: Conference Series (Vol. 1192, No. 1, p. 012018). IOP Publishing
- [3] Amer Abdulmajeed Abdulrahman , Mahmood Khalel Ibrahim (2020) **Toward Constructing a Balanced Intrusion Detection Dataset Based on CICIDS2017** ISSN:2663-7405 Samarra J. Pure Appl. Sci., 2020; 2 (3): 132-142
- [4] Venkata Ramani Varanasi, Shaik Razia (2020) **A Comparative Evaluation of supervised and unsupervised algorithms for Intrusion Detection** International Journal of Advanced Trends in Computer Science and Engineering Volume 9, No. 4, July- August 2020 Available Online : [ijatcse93942020.pdf](http://ijatcse93942020.pdf)
- [5] Yan Bing Ho , Wun-She Yap, Kok-Chin Khor .(2021) **The Effect of Sampling Methods on the CICIDS2017 Network Intrusion Data Set** . online - <https://www.researchgate.net/publication/355117540>
- [6] **CICIDS 2017 Data Set**[Online]. accessed in June 2020. Available: <https://www.unb.ca/cic/datasets/ids-2017.html>
- [7] Tesfahun A and Bhaskari D L 2013 **Intrusion detection using random forests classifier with smote and feature\_reduction** 2013 International Conference on Cloud Ubiquitous Computing Emerging Technologies pp 127 – 132
- [8] Apichit Pattawaro, Chantri Polprasert. (2018). **Anomaly-Based Network Intrusion Detection System through Feature Selection and Hybrid Machine Learning Technique**. 2018, 16th International Conference on ICT and Knowledge Engineering (ICT&KE), Bangkok, Thailand. <https://doi.org/10.1109/ictke.2018.8612331>
- [9] Safura A. Mashayak, Balaji R. Bombade. (2019). **Network Intrusion Detection Exploitation Machine Learning Strategies with the Utilization of Feature Elimination Mechanism**. *International Journal of Computer Sciences and Engineering* Vol. 7(5), May 2019, E-ISSN: 2347-2693. <https://doi.org/10.26438/ijcse/v7i5.12921300>
- [10] Manjula C. Belavagi and Balachandra Muniyal. (2016) **Performance Evaluation of Supervised Machine Learning Algorithms for Intrusion Detection**. *Procedia Computer Science, Volume 89, 2016, Pages 117-123*.
- [11] Ripon Patgiri, Udit Varshney, Tanya Akutota, and Rakesh Kunde. (2018) **An Investigation on Intrusion Detection System Using Machine Learning**. 2018 IEEE Symposium Series on Computational Intelligence (SSCI), Bangalore, India, <https://doi.org/10.1109/ssci.2018.8628676>

- [12] Shaik Razia, M. Bhanusri, **Maize Leaf Disease Detection Using Convolution Neural Network**, , *Jour of Adv Research in Dynamical & Control Systems* 12 (02), 1154
- [13] Md Al-Imran , Shamim H. Ripon (2021) **Network Intrusion Detection: An Analytical Assessment Using Deep Learning and State-of-the-Art Machine Learning Models** *International Journal of Computational Intelligence Systems* 2021) 14:200 Available : <https://doi.org/10.1007/s44196-021-00047-4>
- [14] He, H., Bai, Y., Garcia, E. A., & Li, S. A. (2008). Adasyn: **Adaptive synthetic sampling approach for imbalanced learning**. In *IEEE International Joint Conference on Computational Intelligence, IJCNN* (pp.1322–1328). IEEE.
- [15] Sharafaldin, I.; Lashkari, A.H.; Ghorbani, A.A. (2018). **Toward generating a new intrusion detection dataset and intrusion traffic characterization**. *Proceedings of the Fourth International Conference on Information Systems Security and Privacy, ICISSP*, Funchal, Madeira, Portugal, 22–24 January 2018. <https://doi.org/10.5220/0006639801080116>
- [16] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. (2002) **Smote: Synthetic minority over-sampling technique**. *Journal of Artificial Intelligence Research*, 16 (2002):321-357
- [17] Dina Elreedy , Amir F. Atiya, Firuz Kamalov (2023) .**A theoretical distribution analysis of synthetic minority oversampling technique (SMOTE) for imbalanced learning**. *Machine Learning* (2024) 113:4903–4923. Springer . <https://doi.org/10.1007/s10994-022-06296-4>
- [18] Alex X. Wang, Stefanka S. Chukova, Binh P. Nguyen .(2023) **Synthetic minority oversampling using edited displacement-based  $k$ -nearest neighbors**. *Applied Soft Computing Journal* 148 (2023)110895. <https://doi.org/10.1016/j.asoc.2023.110895>
- [19] Nitesh V. Chawla , Alberto Fernández, Salvador García, Francisco Herrera .(2018). **SMOTE for Learning from Imbalanced Data: Progress and Challenges, Marking the 15-year Anniversary**. *Journal of Artificial Intelligence Research* 61 (2018) 863-905
- [20] Han, H., Wang, W.-Y., & Mao, B.-H. (2005). **Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning**. In *Advances in intelligent computing* (pp.878–887). Berlin, Heidelberg: Springer Berlin Heidelberg
- [21] He, H., Bai, Y., Garcia, E., & Li, S. (2008). **ADASYN: Adaptive synthetic sampling approach for imbalanced learning**. In *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)*:vol. 10, (pp.1322–1328). IEEE.
- [22] Batista, G. E. A. P. A., Prati, R. C., & Monard, M. C. (2004). **A study of the behavior of several methods for balancing machine learning training data**. *SIGKDD Explorations*, 6(1), 20–29.
- [23] Emrah Tufan, Changr Tezcan , And Cengz Acartürk . (2021) **Anomaly-Based Intrusion Detection by Machine Learning: A Case Study on Probing Attacks to an Institutional Network**. *Digital Object Identifier* 10.1109/ACCESS.2021.3068961. IEEE

- [24] Ahmed Tamer Assya, Yahia Mostafaa, Ahmed Abd El-khaleqa, Maggie Mashaly . (2023) **Anomaly-Based Intrusion Detection System using One-Dimensional Convolutional Neural Network**. Procedia Computer Science 220 (2023) 78–85
- [25] D. K. Bhattacharyya and J. K. Kalita, (2013). **Network Anomaly Detection:A Machine Learning Perspective**. Boca Raton, FL, USA: CRC Press.  
[https://www.researchgate.net/publication/307936101\\_Network\\_Anomaly\\_Detection\\_A\\_Machine\\_Learning\\_Perspectiv](https://www.researchgate.net/publication/307936101_Network_Anomaly_Detection_A_Machine_Learning_Perspectiv)
- [26] Mohammed Almuqati , Fatimah Sidi ,Siti Nurulain Mohd Rum ,Maslina Zolkepli ,Challenges in Supervised and Unsupervised Learning: A Comprehensive Overview. International Journal on Advanced Science Engineering and Information Technology · August 2024 . Vol.14 (2024) No. 4  
 ISSN: 2088-5334 <https://www.researchgate.net/publication/383621844>
- [27] K. Moh, M. Aung, and N. N. Oo,(2015) **Association rule pattern mining approaches network anomaly detection**,in Proc. Int. Conf. Future Com-put. Technol., Singapore, 2015, pp. 164-170.
- [28] Hongpo Zhang , Lulu Huang , Chase Qishi Wu , Zhanbo Li , (2020). **An Effective Convolutional Neural Network Based on SMOTE and Gaussian Mixture Model for Intrusion Detection in Imbalanced Dataset**. Computer Networks Volume 177, 4 August 2020, 107315 . online - <https://www.sciencedirect.com/science/article/abs/pii/S1389128620300712?via%3Dihub>
- [29] Razan Abdulhammed, Hassan Musafer, Ali Alessa, Miad Faezipour and Abdelshakour Abuzneid., (2019, **Features Dimensionality Reduction Approaches for Machine Learning Based Network Intrusion Detection**. Electronics 8(3):322; DOI: 10.3390/electronics8030322.
- [30] Laith Alzubaidi , Jinglan Zhang, Amjad J. Humaidi, Ayad Al-Dujaili, Ye Duan, Omran Al-Shamma, J. Santamaria, Mohammed A. Fadhel, Muthana Al-Amidie,Laith Farhan, (2021)**Review of deep learning: concepts, CNN architectures, challenges, applications, future directions** ,Alzubaidi et al. J Big Data (2021) 8:53,Springer ,online- <https://doi.org/10.1186/s40537-021-00444-8> ,
- [31] Mohit Sewak , Sanjay K. Sahay, (2020) , **An Overview of Deep Learning Architecture of Deep Neural Networks and Autoencoders** , Article in Journal of Computational and Theoretical Nanoscience · January 2020, online - <https://www.researchgate.net/publication/339480272>
- [32] Eldad Haber, Lars Ruthotto,(2019),**Stable Architectures for Deep Neural Networks** ,arXiv:1705.03341v3 [cs.LG] 16 Feb 2019
- [33] Jacques Wainera. (2023) , **An empirical evaluation of imbalanced data strategies from a practitioner’s point of view** , arXiv:1810.07168v2 [cs.LG] 10 Nov 2023
- [34] Mahesh T R , Vinoth Kumar , Dhilip Kumar , Oana Geman , Martin Margala ,Manisha Guduri .(2023) **The stratified K-folds cross-validation and class-balancing methods with high-performance ensemble classifiers for breast cancer classification**. Published by Elsevier Inc. <https://doi.org/10.1016/j.health.2023.100247>
- [35] Xiaoguang Yuan , Shiruo Liu , Wei Feng and Gabriel Dauphin.(2023).**Feature Importance Ranking of Random Forest-Based End-to-End Learning Algorithm**.Remote Sens. 2023, 15(21), 5203.  
<https://doi.org/10.3390/rs15215203>

- [36] Gerstorfer, Y, Hahn-Klimroth, M and Krieg, L. 2023. **A Notion of Feature Importance by Decorrelation and Detection of Trends by Random Forest Regression**. Data Science Journal, 22: 42, pp. 1–14. DOI <https://doi.org/10.5334/dsj-2023-042>
- [37] Breiman L. (2001) Random Forests. **Machine Learning**, 45, 5-32. Springer <http://dx.doi.org/10.1023/A:1010933404324>
- [38] T. Cover , P. Hart ,(1967) , **Nearest neighbor pattern classification**, IEEE Transactions on Information Theory ( Volume: 13, Issue: 1, January 1967)
- [39] Hosmer Jr., D.W., Lemeshow, S. and Sturdivant, R.X. (2013) **Applied Logistic Regression**. 3rd Edition, John Wiley & Sons, Hoboken, NJ. <https://doi.org/10.1002/9781118548387>
- [40] John, G.H. and Langley, P. (1995) **Estimating Continuous Distributions in Bayesian Classifiers**. Proceedings of the 11th Conference on Uncertainty in Artificial Intelligence, Montreal, 18-20 August 1995, 338-345.
- [41] Jain, A.K. (2010) **Data Clustering: 50 Years beyond K-Means**. Pattern Recognition Letters Volume 31, Issue 8, 1 June 2010, Pages 651-666. <http://dx.doi.org/10.1016/j.patrec.2009.09.011>
- [42] Bellman R. (1961). **Adaptive control processes**. Princeton, NJ: Princeton University Press.
- [43] Kaufman, L., & Rousseeuw, P. J. (1990). **Finding Groups in Data An Introduction to Cluster Analysis**. New York: John Wiley & Sons Inc. <https://doi.org/10.1002/9780470316801>
- [44] Fei Tony Liu,Zhi-Hua Zhou,Kai Ming Ting ,(2008) **Isolation-Based Anomaly Detection** , ACM Transactions on Knowledge Discovery from Data 6(1):1-39 , DOI:10.1145/2133360.2133363
- [45] Kay Henning Brodersen; Cheng Soon Ong; Klaas Enno Stephan; Joachim M. Buhmann, (2010).**The Balanced Accuracy and Its Posterior Distribution**.IEEE 20th International Conference on Pattern Recognition. [10.1109/ICPR.2010.764](https://doi.org/10.1109/ICPR.2010.764)
- [46] Marina Sokolova , Guy Lapalme.(2009).**A systematic analysis of performance measures for classification tasks**.Information Processing & Management Volume 45,Pages 427 437. <https://doi.org/10.1016/j.ipm.2009.03.002>
- [47] Varun Chandola, Arindam Banerjee, Vipin Kumar.(2009)Anomaly detection: A survey.ACM Computing Surveys (CSUR), Volume 41, Issue 3. <https://doi.org/10.1145/1541880.1541882>
- [48] Davide Chicco , Giuseppe Jurman.(2020).**The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation**.BMC Genomics volume 21, Article number: 6.
- [49] B.W. Matthews.(1975).**Comparison of the predicted and observed secondary structure of T4 phage lysozyme**.Biochimica et Biophysica Acta (BBA) - Protein Structure Volume 405, Issue 2, 20 October 1975, Pages 442-451.[https://doi.org/10.1016/0005-2795\(75\)90109-9](https://doi.org/10.1016/0005-2795(75)90109-9)

- [50] Bishop C. M. (2006). **Pattern Recognition and Machine Learning**. Springer.
- [51] Hosmer, D. W., Lemeshow, S. (2000). **Applied Logistic Regression**. Wiley.
- [52] Zheng Wang , Yang Li, Li-Ping Li , Zhu-Hong You , and Wen-Zhun Huang .(2021).**Self-Interacting Proteins Prediction from PSSM Based on Evolutionary Information**.Hindawi Scientific Programming Volume 2021, Article ID 6677758, 10 pages  
<https://doi.org/10.1155/2021/6677758>
- [53] FAISAL SHEHZAD, MUHAMMAD RASHID, MOHAMMED H SINKY, SAUD S ALOTAIBI, MUHAMMAD YOUSUF IRFAN ZIA.(2021).**A Scalable System-on-Chip Acceleration for Deep Neural Networks**.DOI 10.1109/ACCESS.2021.3094675, IEEE Access.
- [54] Bergstra, J., & Bengio, Y. (2012).**Random Search for Hyper-Parameter Optimization**.Journal of Machine Learning Research (JMLR), 13, 281-305.
- [55] David W Hosmer ,Stanley Lemeshow ,Rodney X Sturdivant.(2013).**The Multiple Logistic Regression Model. Applied Logistic Regression**, Third Edition (pp.35-47). DOI:10.1002/9781118548387.ch2